



School of Artificial Intelligence
Jilin University

第五章 动态规划

吉林大学人工智能学院

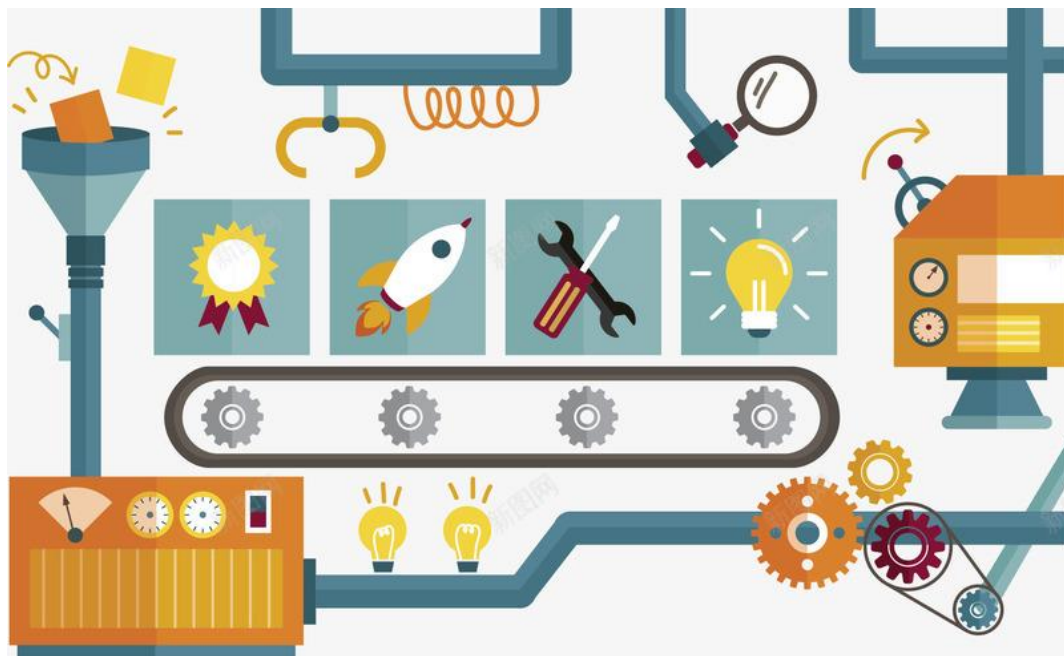
王元宏

- 5.1 0-1背包问题
- 5.2 动态规划主要思想
- 5.3 再论背包问题
- 5.4 可靠性设计
- 5.5 最短路径问题
- 5.6 再论货郎担问题
- 5.7 最长公共子序列
- 5.8 编辑距离问题
- 5.9 最优二分检索树
- 5.10 矩阵链乘积问题

- 5.1 0-1背包问题
- 5.2 动态规划主要思想
- 5.3 再论背包问题
- **5.4 可靠性设计**
- 5.5 最短路径问题
- 5.6 再论货郎担问题
- 5.7 最长公共子序列
- 5.8 编辑距离问题
- 5.9 最优二分检索树
- 5.10 矩阵链乘积问题

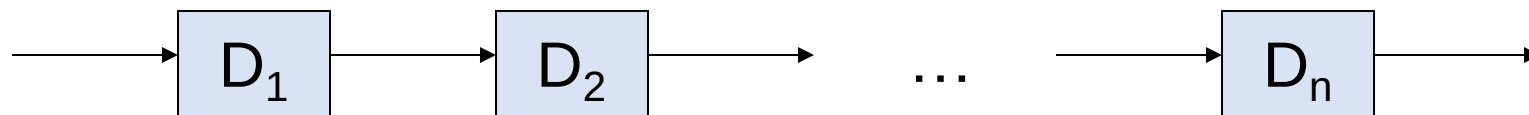
问题描述

- 工厂流水线由若干设备以串联方式连接在一起
- 每个设备都有一定概率出故障，正常运转的概率即为设备的可靠性
- 整个系统的可靠性为每个系统可靠性的乘积



问题描述

- 工厂流水线由若干设备以串联方式连接在一起
- 每个设备都有一定概率出故障，正常运转的概率即为设备的可靠性
- 整个系统的可靠性为每个系统可靠性的乘积

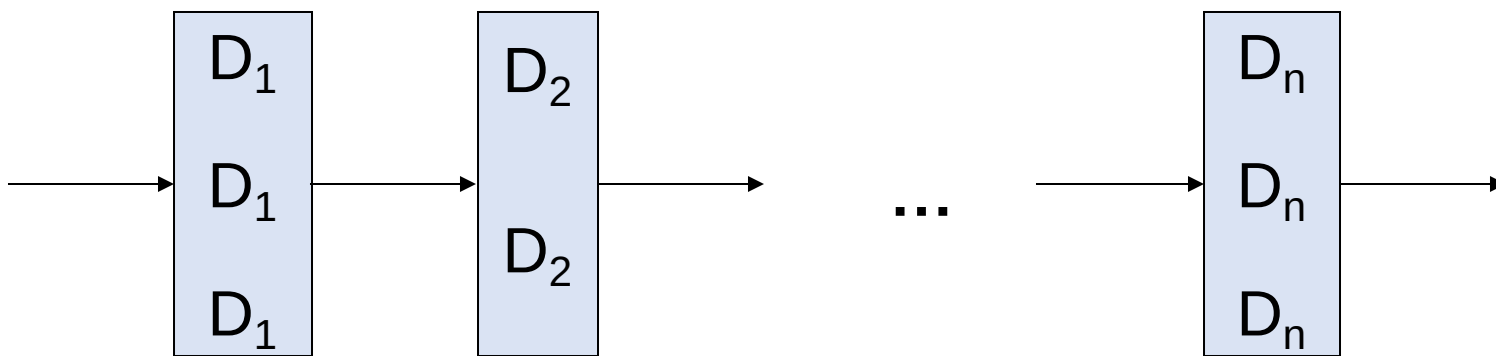


- r_i 为设备 D_i 的可靠性
- 那么整个系统的可靠性 = $\prod r_i$

0.1的影响力! $0.9 \times 0.9 \times \cdots \times 0.9 = 0.9^{20} \approx 0.12$

问题：如何提高整个系统的可靠性？

- 增加一些冗余的设备

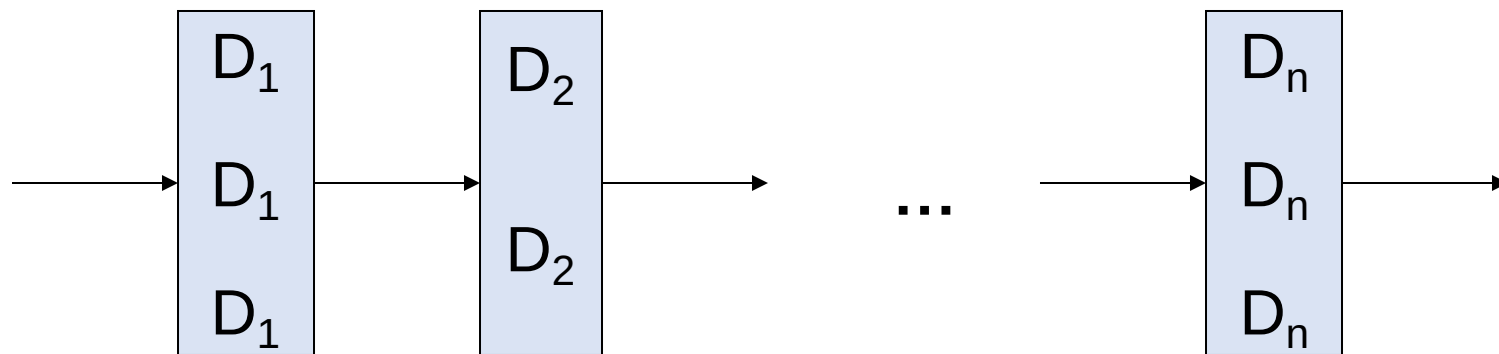


- 假设设备 D_i 的台数为 x_i (冗余了 $x_i - 1$ 台), 那么它们同时出现故障的概率为 $(1 - r_i)^{x_i}$, 可靠性为 $1 - (1 - r_i)^{x_i}$

0.1的影响力! $1 - (1 - 0.9)^{20} = 1 - 10^{-20}$

问题：如何通过冗余设备尽可能提高整个系统的可靠性？

- 输入：串联系统的级数 n ，每级设备的可靠性 r_i ，成本 p_i ，最大成本 C
- 输出：每级设备配置的台数 x_i ，使得整个系统的可靠性最大



- 输入：串联系统的级数 n ，每级设备的可靠性 r_i ，成本 p_i ，最大成本 C
- 输出：每级设备配置的台数 x_i ，使得整个系统的可靠性最大
- 若 i 级设备配置了 x_i 台，那么可靠性为 $\phi_i(x_i) = 1 - (1 - r_i)^{x_i}$
- 系统总的可靠性为 $\prod_{i=1}^n \phi_i(x_i)$
- 每级设备至少需要配置一台
- 第 j 级至多配置的台数为 $u_j = \lfloor (C - \sum_{i \neq j} p_i) / p_j \rfloor$

$$\max_{x_1, x_2, \dots, x_n \in \mathbb{N}} \prod_{i=1}^n \phi_i(x_i)$$

优化目标

$$s. t. \sum_{i=1}^n x_i p_i \leq C$$

约束条件

$$1 \leq x_i \leq u_i$$

转换为多重背包问题

- 输入：
 - n 种商品组成的集合 O ，每种商品的个数为 k_i ，有两个属性 v_i 和 p_i ，表示体积和价格
 - 背包容量 C
- 输出：
 - 求解放入背包的商品个数 $x_1, x_2, \dots, x_n \in \mathbb{N}$, 令

$$\begin{aligned} & \max_{x_1, x_2, \dots, x_n \in \mathbb{N}} \prod_{i=1}^n \phi_i(x_i) \\ & s.t. \sum_{i=1}^n x_i p_i \leq C \\ & \quad 1 \leq x_i \leq u_i \end{aligned}$$

$$\begin{aligned} & \max \sum_{i \in S} x_i p_i \\ & s.t. \sum_{i \in S} x_i v_i \leq C \text{ and } x_i \leq k_i \end{aligned}$$

优化目标

约束条件

思考：是否可以将可靠性问题转换成多重背包问题？

转换为多重背包问题

$$\begin{array}{l} \max_{x_1, x_2, \dots, x_n \in \mathbb{N}} \prod_{i=1}^n \phi_i(x_i) \\ \text{s.t. } \sum_{i=1}^n x_i p_i \leq C \\ 1 \leq x_i \leq u_i \end{array} \longrightarrow \begin{array}{l} \max \sum_{i \in S} x_i p_i \\ \text{s.t. } \sum_{i \in S} x_i v_i \leq C \text{ and } x_i \leq k_i \end{array}$$

- 多重背包递推公式: $P[i, c] = \max_{k=0}^{k_i} \{P[i-1, c - kv_i] + kp_i\}$
- 令 $F[i, c]$ 为在成本 c 的约束下对前 i 级设备配置的最大可靠性
- 递推公式: $F[i, c] = \max_{k=1}^{u_i} \{F[i-1, c - kp_i] \cdot \phi_i(k)\}$

可靠性问题的序偶对法

- 递推公式: $F[i, c] = \max_{k=1}^{+\infty} \{F[i-1, c - kp_i] \cdot \phi_i(k)\}$
- 函数表示: $f_i(x) = F[i, x]$
- 函数递推式:

$$f_i(x) = \max_{k=1}^{u_i} \{f_{i-1}(x - kp_i) \cdot \phi_i(k)\}$$

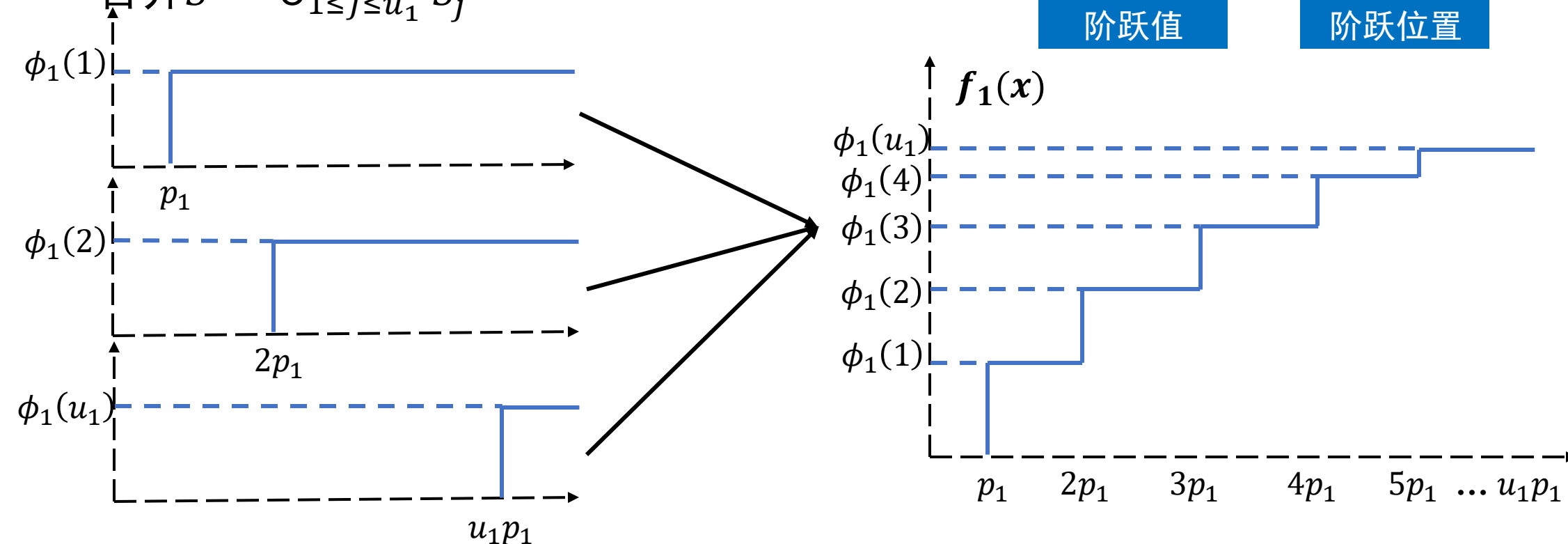
- 可以采用类似0-1背包的序偶对法求解

可靠性问题的序偶对法

- 函数递推式:

$$f_i(x) = \max_{k=1}^{n=u_i} \{f_{i-1}(x - kp_i) \cdot \phi_i(k)\}$$

- $S^0 = \{(1, 0)\}$
- $1 \leq j \leq u_1, S_j^1 = \{(\phi_1(j) \cdot f, x + j \cdot p_1) \mid (f, x) \in S^0\} = \{(\phi_1(j), j \cdot p_1)\}$
- 合并 $S^1 = \bigcup_{1 \leq j \leq u_1} S_j^1$



- 函数递推式：

$$f_i(x) = \max_{k=1}^{n=u_i} \{f_{i-1}(x - kp_i) \cdot \phi_i(k)\}$$

- $S^0 = \{(1, 0)\}$
- S^i ：配置前 i 级设备最优可靠性函数的序偶集合
- S_j^i ：配置第 i 级设备为 j 台时，最优序偶可靠性函数的序偶集合
- 基于 S^{i-1} 构造 S_j^i ：

$$S_j^i = \{(\phi_i(j) \cdot f, x + j \cdot p_i) \mid (f, x) \in S^{i-1}\}$$

- 合并 S_j^i 得到 S^i ：

$$S^i = \bigcup_{1 \leq j \leq u_i} S_j^i$$

思考：如何定义支配规则？

可靠性问题的序偶对法

- 函数递推式：

$$f_i(x) = \max_{k=1}^{n=u_i} \{f_{i-1}(x - kp_i) \cdot \phi_i(k)\}$$

- $S^0 = \{(1, 0)\}$
- S^i ：配置前 i 级设备最优可靠性函数的序偶集合
- S_j^i ：配置第 i 级设备为 j 台时，最优序偶可靠性函数的序偶集合
- 基于 S^{i-1} 构造 S_j^i ：

$$S_j^i = \{(\phi_i(j) \cdot f, x + j \cdot p_i) \mid (f, x) \in S^{i-1}\}$$

- 合并 S_j^i 得到 S^i ：

$$S^i = \bigcup_{1 \leq j \leq u_i} S_j^i$$

支配规则：对于 (f_1, x_1) 和 (f_2, x_2) ，当且仅当 $f_1 \geq f_2$ 而 $x_1 \leq x_2$ 时， (f_1, x_1) 支配 (f_2, x_2) 。
 (f_2, x_2) 从序偶集合中舍去。

- $S^0 = \{(1, 0)\}$
- S^i : 配置前 i 级设备最优可靠性函数的序偶集合
- S_j^i : 配置第 i 级设备为 j 台时, 最优序偶可靠性函数的序偶集合

```
对于  $i=1, 2, \dots, n$ , 求解  $u_i$   
 $S^0 \leftarrow \{(1, 0)\}$   
for  $i \leftarrow 1$  to  $n$   
    for  $j \leftarrow 1$  to  $u_i$   
        求  $\phi_i(j)$   
        求  $S_j^i$   
    end  
    基于支配规则归并  $S^i$   
end
```

- 设计一个由设备D1, D2, D3组成的三级系统。
 - 每台设备的成本分别为30元, 15元和20元
 - 可靠性分别是0.9, 0.8, 0.5
 - 计划建立该系统的投资不得超过105元
- $C=105$, $p_1=30$, $p_2=15$, $p_3=20$, $r_1=0.9$, $r_2=0.8$, $r_3=0.5$

- $C=105$, $p_1=30$, $p_2=15$, $p_3=20$, $r_1=0.9$, $r_2=0.8$, $r_3=0.5$
- $S_0=\{(1,0)\}$
- $u_1 = (105-15-20)/30 = 2$, $u_2=3$, $u_3=3$
- $u_1 = 2$, $p_1=30$, $r_1=0.9$,
 $\phi_1(1)=0.9$, $\phi_1(2)=1-(1-0.9)^2=0.99$
 $S^1_1=\{(0.9,30)\}$, $S^1_2=\{(0.99,60)\}$
 $S^1=\{(0.9,30),(0.99,60)\}$
- $u_2 = 3$, $p_2=15$, $r_2=0.8$,
 $\phi_2(1)=0.8$, $\phi_2(2)=1-(1-0.8)^2=0.96$, $\phi_2(3)=1-(1-0.8)^3=0.992$
 $S^2_1=\{(0.9*0.8,30+15), (0.99*0.8, 60+15)\}=\{(0.72, 45),(0.792, 75)\}$
 $S^2_2=\{(0.9*0.96,30+15*2), (0.99*0.96, \text{60+15*2})\}=\{(0.864, 60)\}$
 $S^2_3=\{(0.9*0.992,30+15*3), (0.99*0.992, \text{60+15*3})\}=\{(0.8928,75)\}$
 $S^2=\{(0.72, 45),(0.792, 75),(0.864,60), (0.8928,75)\}$

无法配置一台 D_3

- $S^2 = \{(0.72, 45), (0.864, 60), (0.8928, 75)\}$
- $u_3 = 3, p_3 = 20, r_3 = 0.5,$
 $\phi_3(1) = 0.5, \phi_3(2) = 1 - (1 - 0.5)^2 = 0.75, \phi_3(3) = 1 - (1 - 0.5)^3 = 0.875$
 $S^3_1 = \{(0.72 * 0.5, 45 + 20), (0.864 * 0.5, 60 + 20), (0.8928 * 0.5, 75 + 20)\}$
 $= \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$
 $S^3_2 = \{(0.72 * 0.75, 45 + 20 * 2), (0.864 * 0.75, 60 + 20 * 2), (0.8928 * 0.75, 75 + 20 * 2)\}$
 $= \{(0.54, 85), (0.648, 100)\}$
 $S^3_3 = \{(0.72 * 0.875, 45 + 20 * 3)\} = \{(0.63, 105)\}$
 $S^3 = \{(0.36, 65), (0.432, 80), \text{被支配}, (0.54, 85), (0.648, 100), \text{被支配}\}$

- 由 $S^n, S^{n-1} \dots S^1$ 回溯求 $m_n, m_{n-1}, \dots, m_1$
- 判断由 S^n 中 f 最大的序偶来自哪个 S^n_j , 则 n 级设备上的设备数量为 j , 以此类推:
- $(0.648, 100) \in S^3_2$, 所以 $x_3 = 2$; $(0.864, 60) \in S^2_2$, 所以 $x_2 = 2$; $(0.9, 30) \in S^1_1$, $x_1 = 1$

决策序列 $(x_1, x_2, x_3) = (1, 2, 2)$

- 5.1 0-1背包问题
- 5.2 动态规划主要思想
- 5.3 再论背包问题
- 5.4 可靠性设计
- **5.5 最短路径问题**
- 5.6 再论货郎担问题
- 5.7 最长公共子序列
- 5.8 编辑距离问题
- 5.9 最优二分检索树
- 5.10 矩阵链乘积问题

-

最短路径问题

- 从正新楼到敬信楼，怎么走最近？
- 从正新楼到日新楼，怎么走最近？



最短路径问题

- 从正新楼到敬信楼，怎么走最近？
- 从正新楼到日新楼，怎么走最近？
- 从逸夫楼到敬信楼，怎么走最近？

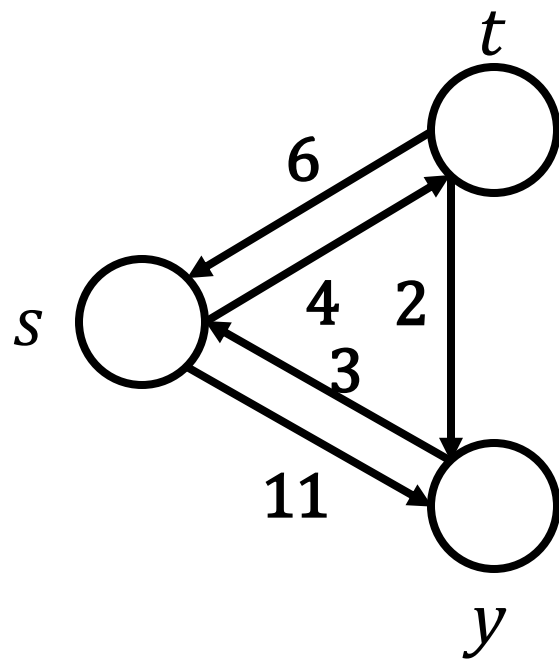


-
- A detailed map of the Changchun University area. The map shows a grid-like street pattern with several key locations labeled. In the top left, "硅谷大街" (Silicon Valley Avenue) runs diagonally. To its right is "修正路" (Zhengxue Road). A red location pin marks a spot near "吉林大学小区" (Jilin University Residential Area) and "文苑二公寓" (Wenyuan No. 2 Apartment). Other labeled areas include "北苑学生一公寓" (North Garden Student No. 1 Apartment), "前卫南区鼎新楼" (Qianwei South District Dingxin Building), "吉林大学前卫南区鼎南广场" (Jilin University Qianwei South District Dingnan Plaza), "吉林大学前卫校区南区-李四光楼" (Jilin University Qianwei Campus South District - Li Si Guang Building), "吉林动漫游戏产业区" (Jilin Animation Game Industry Zone), "垠禄·倚澜观邸" (Yinlu · Yilan Guandip), "友谊会馆" (Youyi Hall), "唐敖庆楼" (Tang Aoqing Tower), "长春市财政" (Changchun City Finance), "迪瑞医疗" (Dirui Medical), "长春普华制药股份有限公司" (Changchun Puhua Pharmaceutical Co., Ltd.), "长春建筑学院(高新校区)" (Changchun Architectural College (High-tech Campus)), "长春健康职业学院" (Changchun Health Vocational College), and "蔚山路" (Weishan Road) at the bottom right. Water bodies are shown in light blue, and green spaces are indicated by green patches.

	正新楼	敬信楼	逸夫楼	
正新楼		1.3 km	0.8 km	
敬信楼	1.3 km		1 km	
逸夫楼	0.7 km	1 km		
.....				

最短路径问题

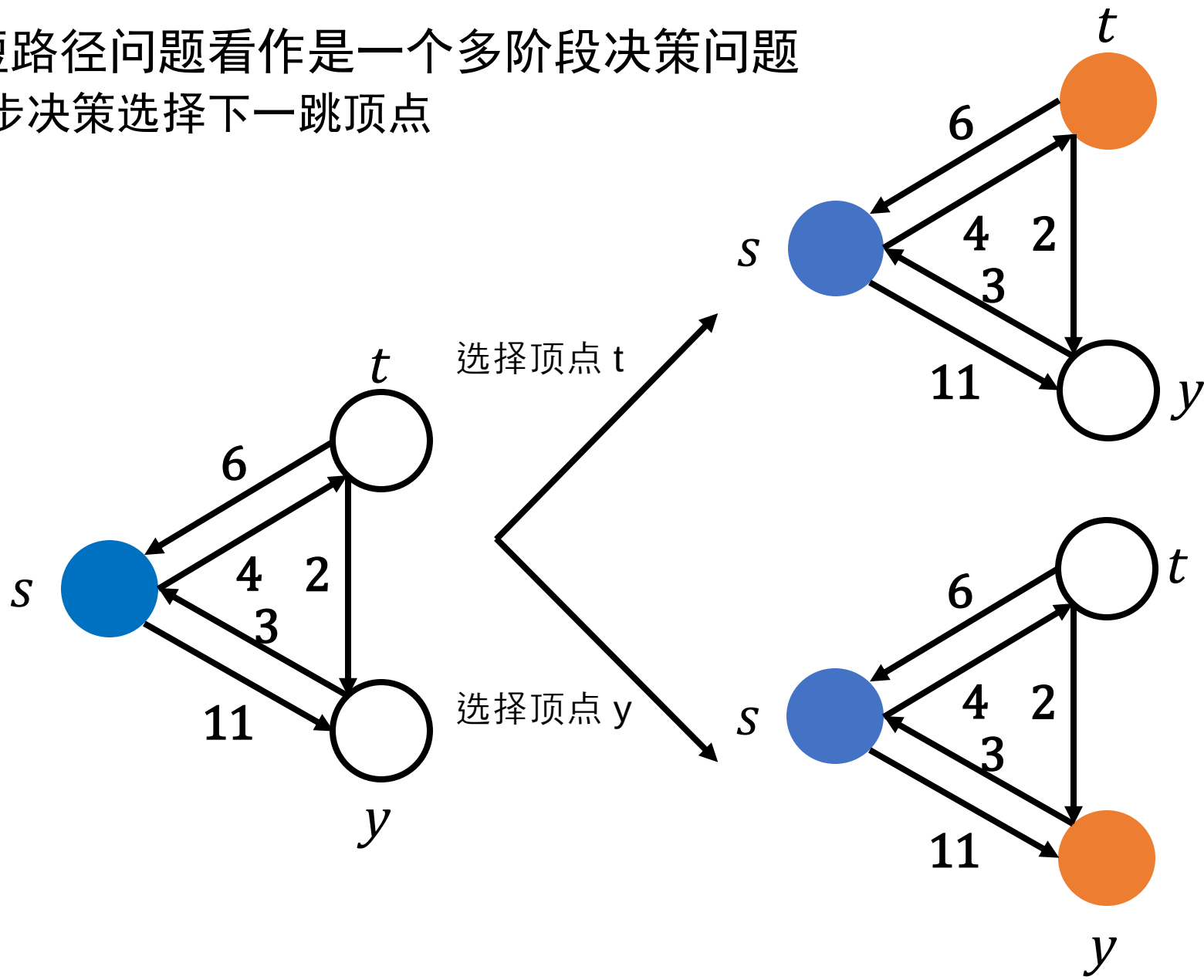
- 输入：
 - 带权图 $G = \langle V, E, W \rangle$, 对于所有的 $(u, v) \in E$, $W(v, u) \geq 0$
- 输出：
 - 从图中任意顶点 u 到 v 的最短距离



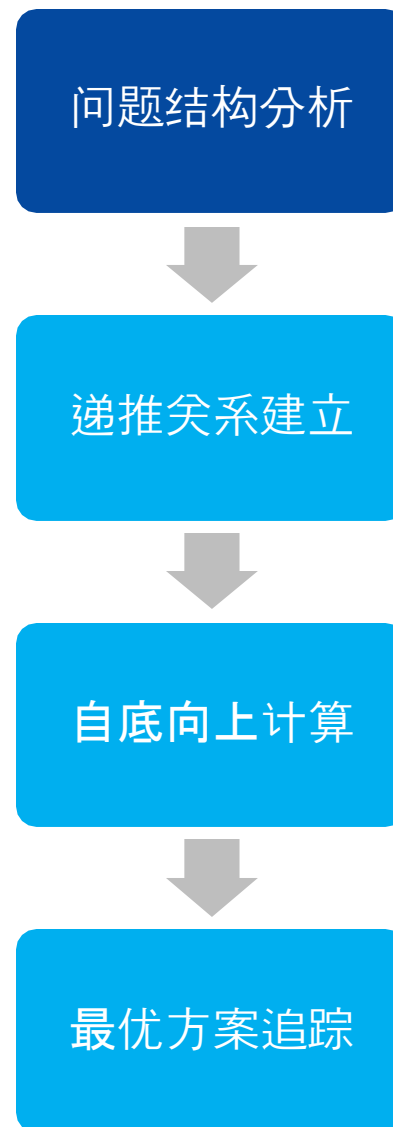
	s	t	y
s	0	4	6
t	5	0	2
y	3	7	0

多阶段决策问题

- 把最短路径问题看作是一个多阶段决策问题
 - 每步决策选择下一跳顶点



- 把最短路径问题看作是一个多阶段决策问题
 - 每步决策选择下一跳顶点
- 令 $dist[u, v]$ 为顶点 u 到 v 的最短路径



- 把最短路径问题看作是一个多阶段决策问题
 - 每步决策选择下一跳顶点
- 令 $dist[u, v]$ 为顶点 u 到 v 的最短路径
- 若 $u \rightarrow v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_m \rightarrow v$ 是一条最短路径
 - 那么 $u \rightarrow v_1 \rightarrow \cdots \rightarrow v_{i-1} \rightarrow v_i$ 一定是 u 到 v_i 的一条最短路径
 - $v_i \rightarrow v_{i+1} \rightarrow \cdots \rightarrow v_m \rightarrow v$ 一定是 v_i 到 v 的一条最短路径
- 满足最优子结构性质
- 递推公式： $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$

问题：如何确定计算顺序？

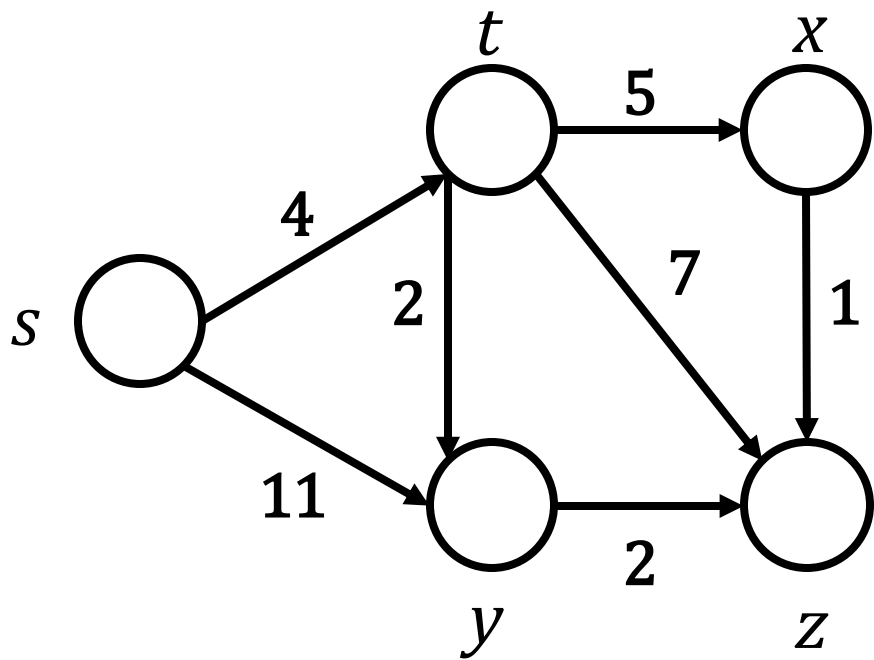
问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序



问题结构分析

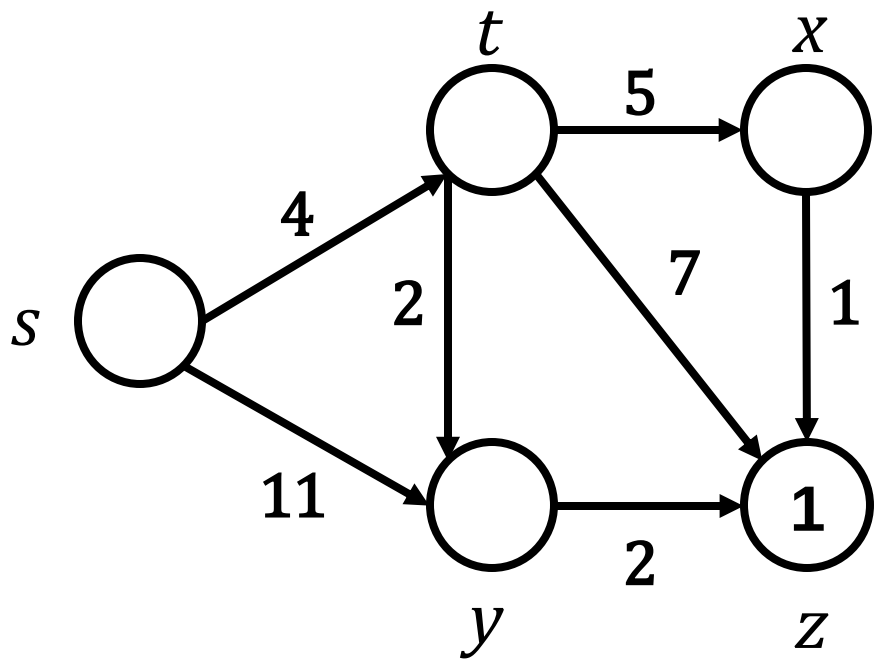
递推关系建立

自底向上计算

最优方案追踪

动态规划求解

- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序



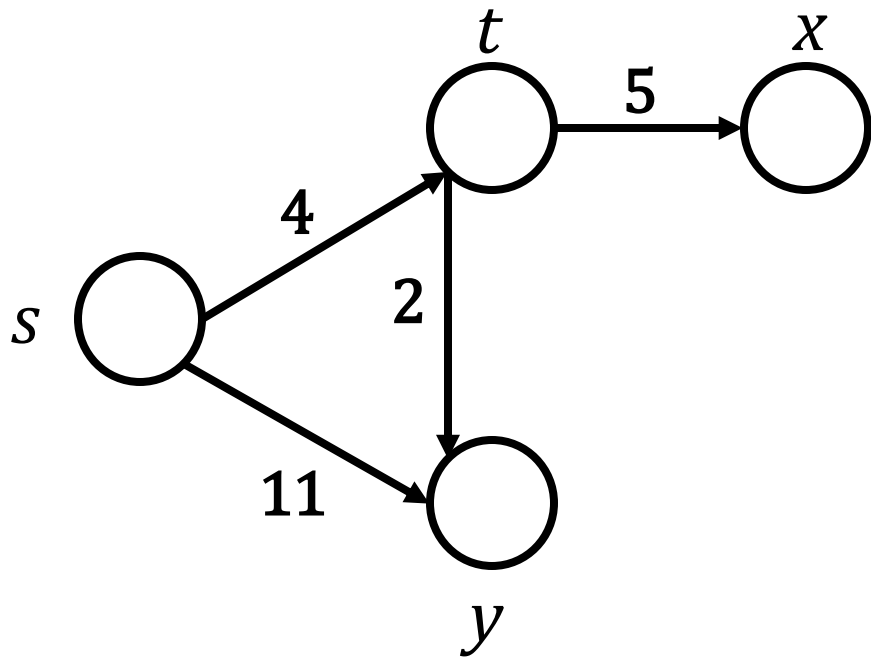
问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序



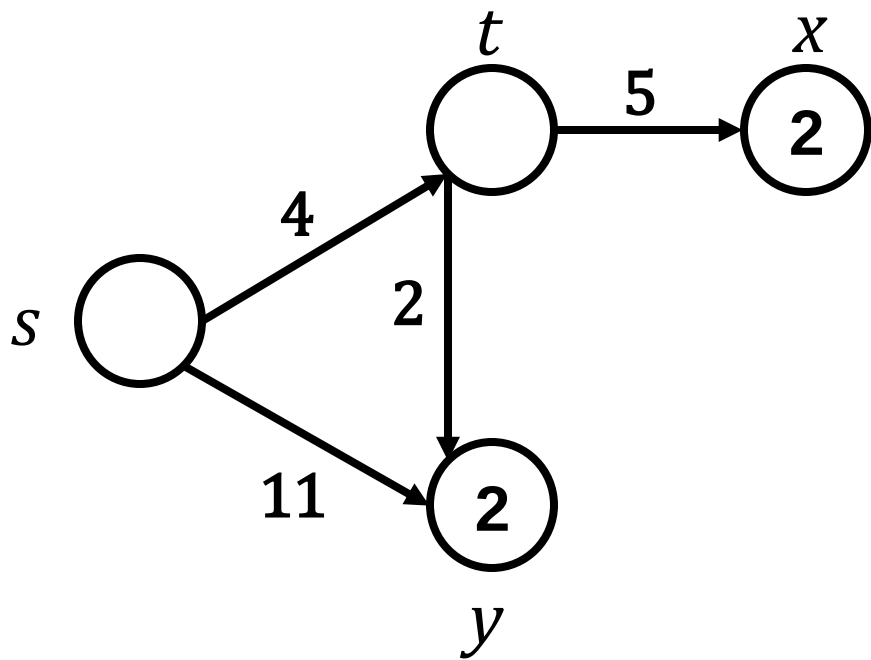
问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序



问题结构分析



递推关系建立

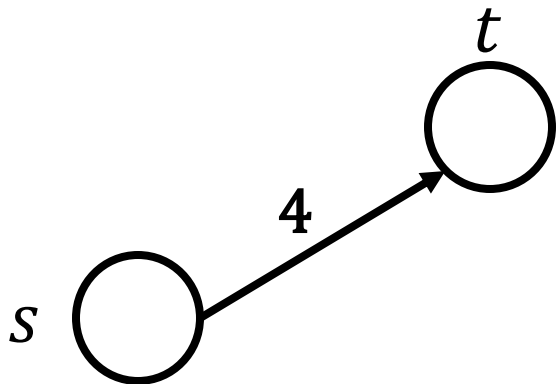


自底向上计算



最优方案追踪

- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序



问题结构分析



递推关系建立

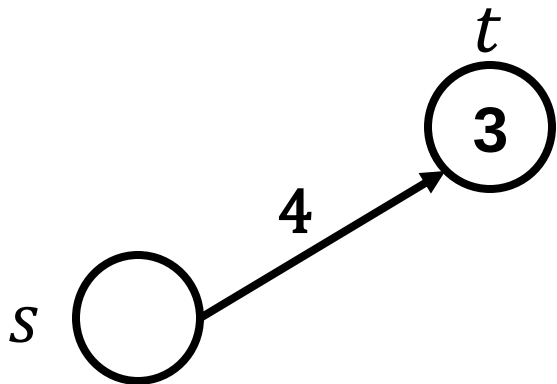


自底向上计算



最优方案追踪

- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序



问题结构分析



递推关系建立

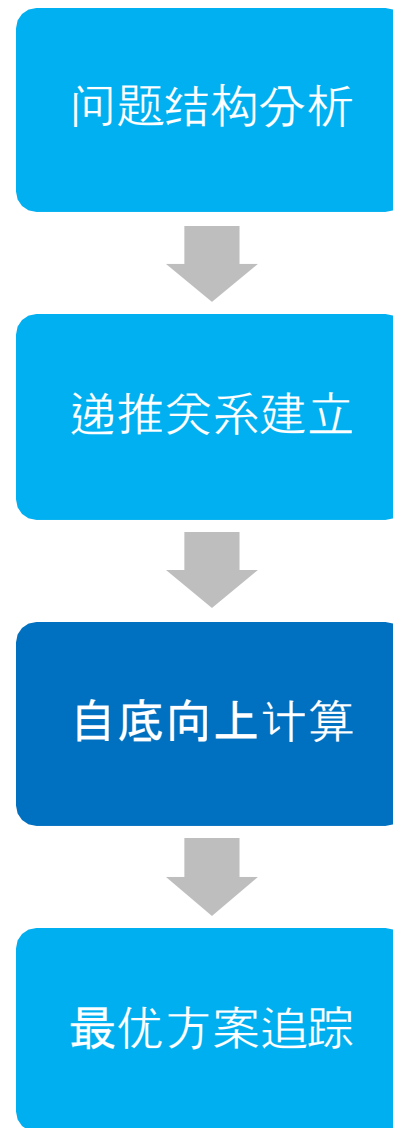


自底向上计算



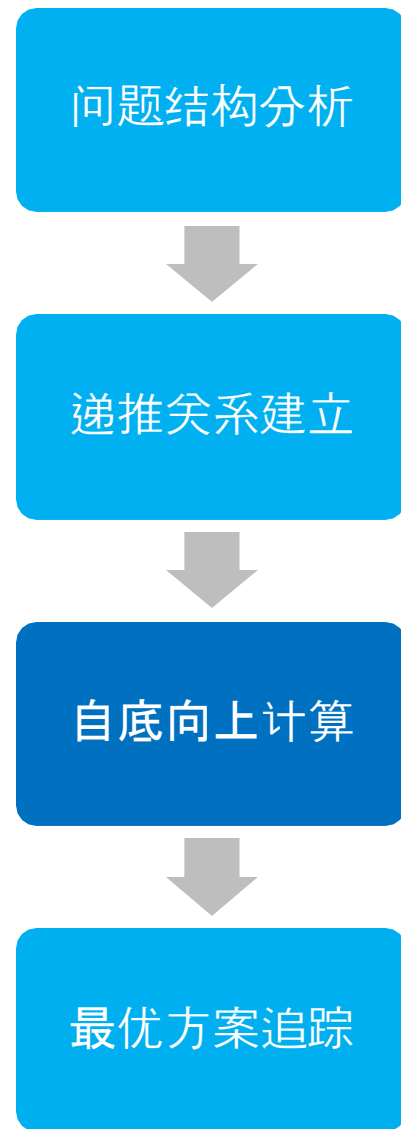
最优方案追踪

- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序

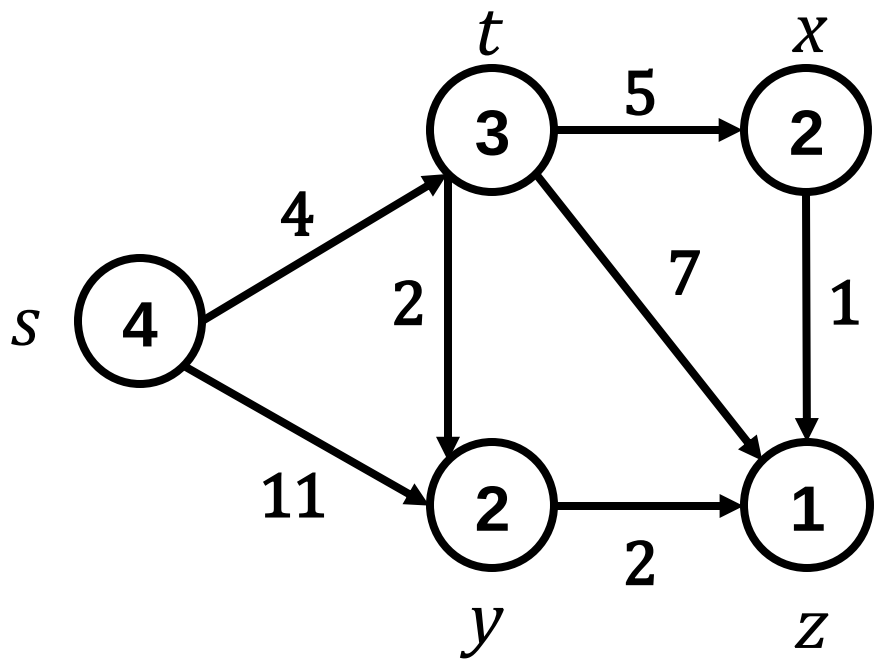


- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序

s (4)



- 如果是有向无环图 (DAG)
- 如何确定计算顺序：拓扑序



问题结构分析

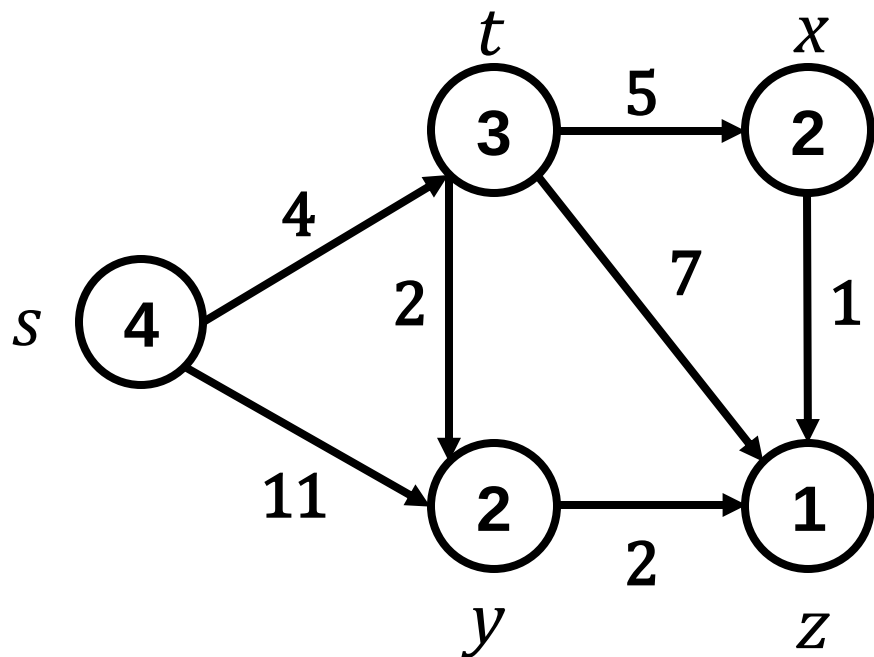
递推关系建立

自底向上计算

最优方案追踪

问题实例

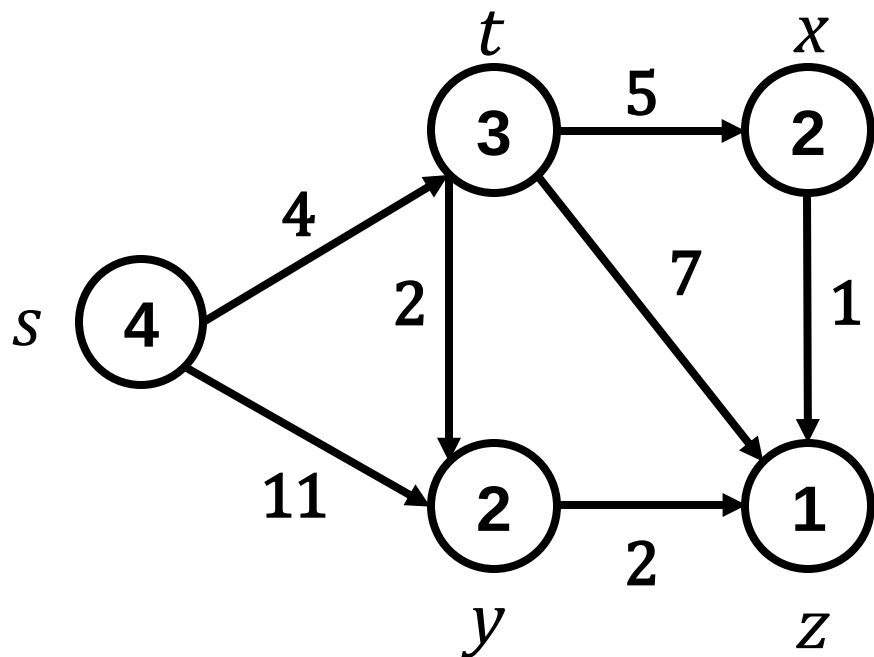
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>					
<i>t</i>					
<i>x</i>					
<i>y</i>					
<i>z</i>					

问题实例

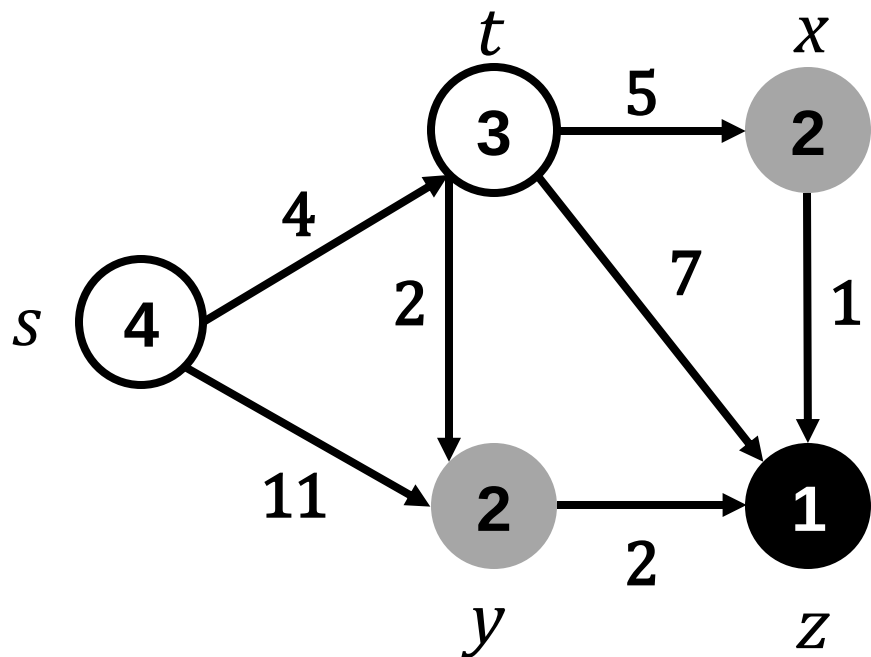
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0			
<i>x</i>	∞	∞	0		
<i>y</i>	∞	∞	∞	0	
<i>z</i>	∞	∞	∞	∞	0

问题实例

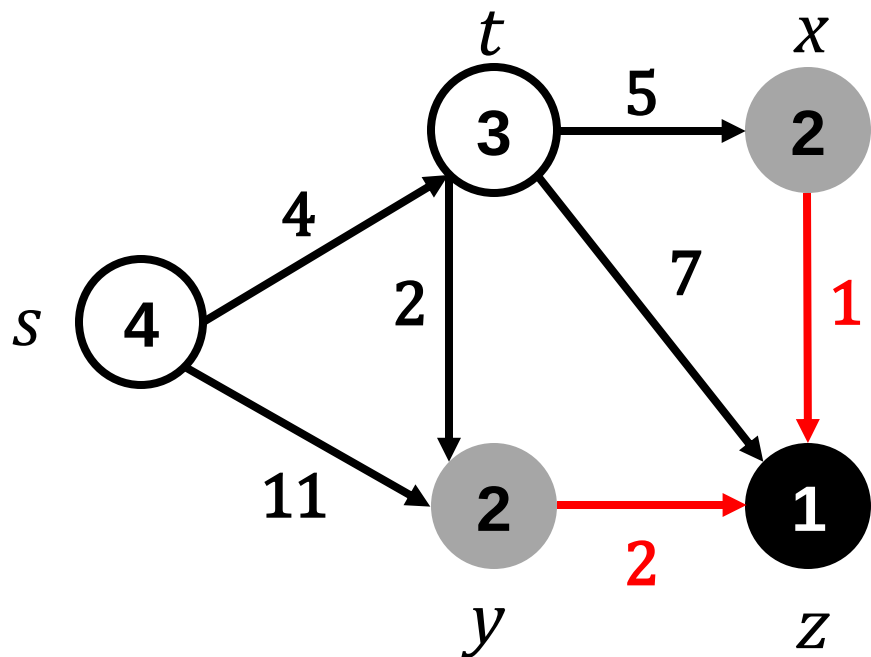
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0			
<i>x</i>	∞	∞	0		
<i>y</i>	∞	∞	∞	0	
<i>z</i>	∞	∞	∞	∞	0

问题实例

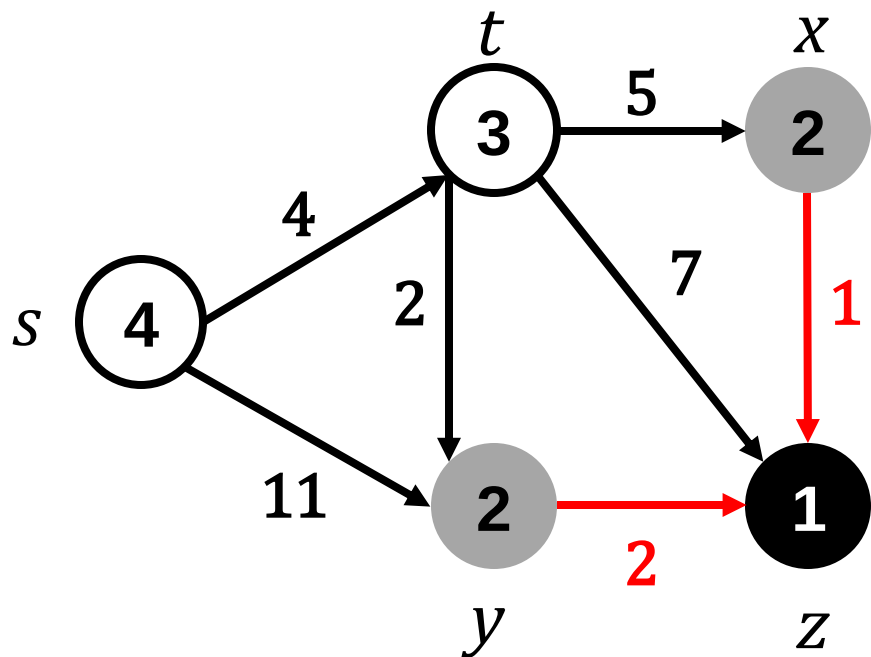
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0			
<i>x</i>	∞	∞	0		
<i>y</i>	∞	∞	∞	0	
<i>z</i>	∞	∞	∞	∞	0

问题实例

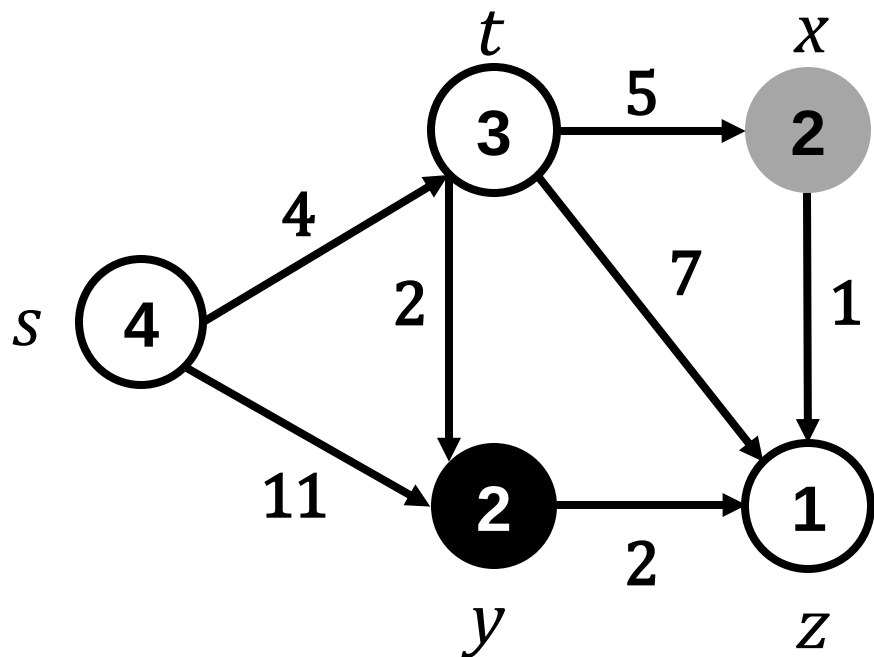
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0			
<i>x</i>	∞	∞	0		1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

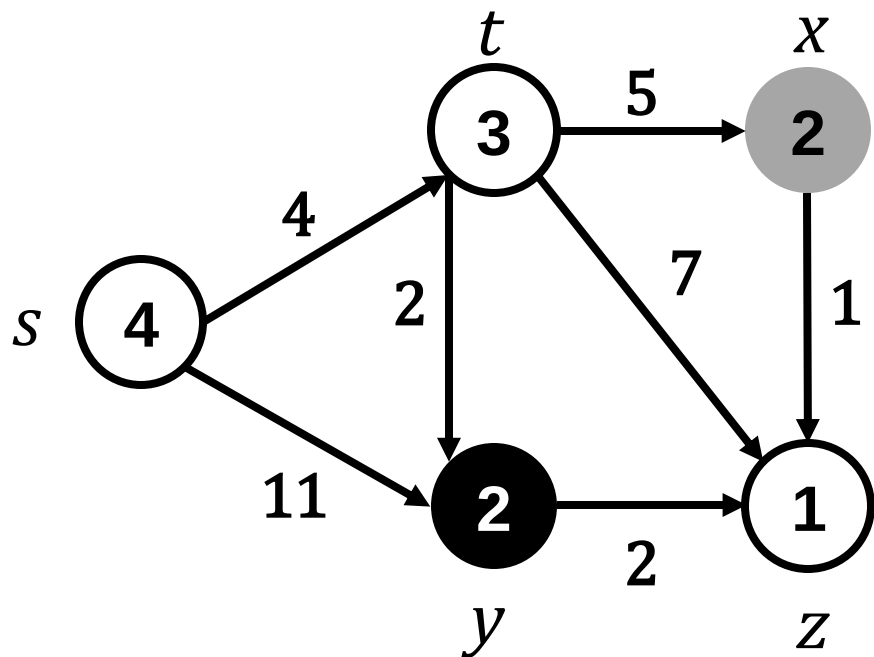
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0			
<i>x</i>	∞	∞	0		1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

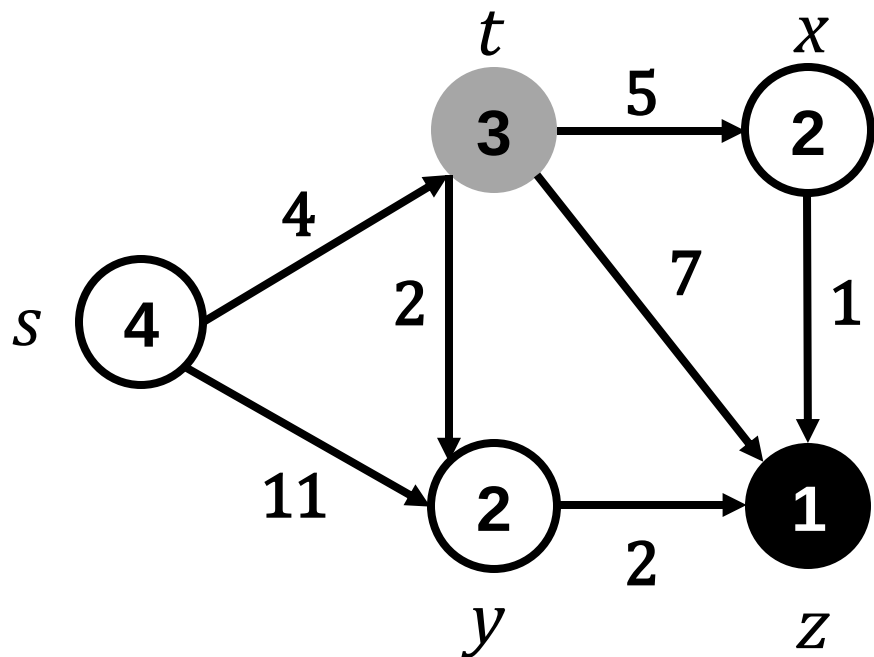
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0			
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

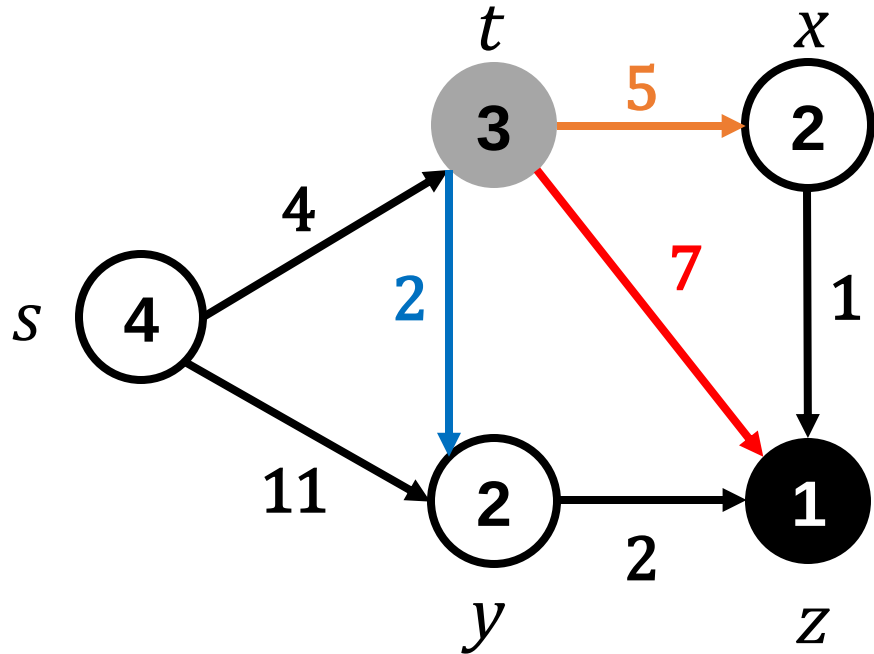
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0			
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

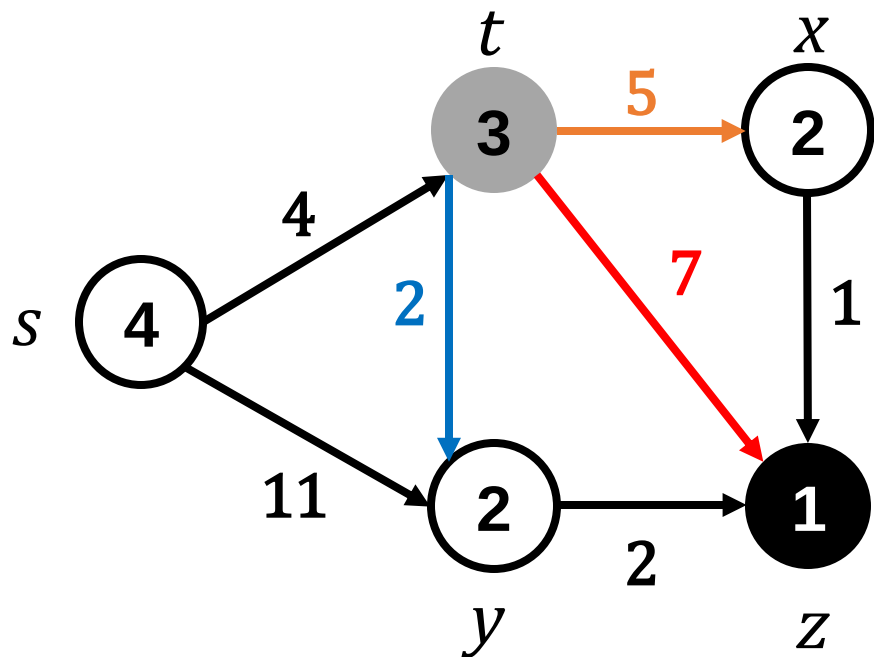
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



$dist$	s	t	x	y	z
s	0				
t	∞	0			
x	∞	∞	0	∞	1
y	∞	∞	∞	0	2
z	∞	∞	∞	∞	0

问题实例

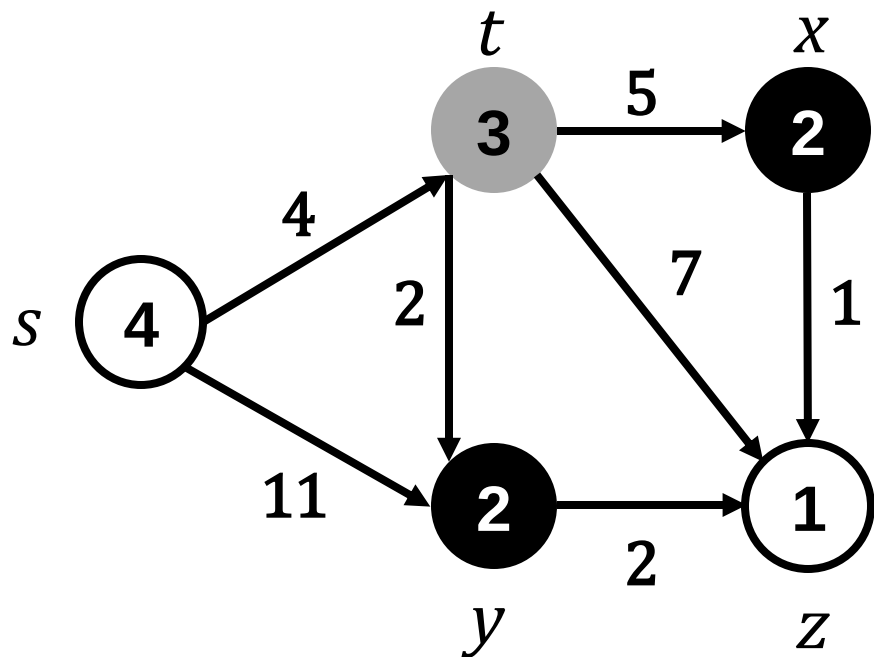
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



$dist$	s	t	x	y	z
s	0				
t	∞	0			4
x	∞	∞	0	∞	1
y	∞	∞	∞	0	2
z	∞	∞	∞	∞	0

问题实例

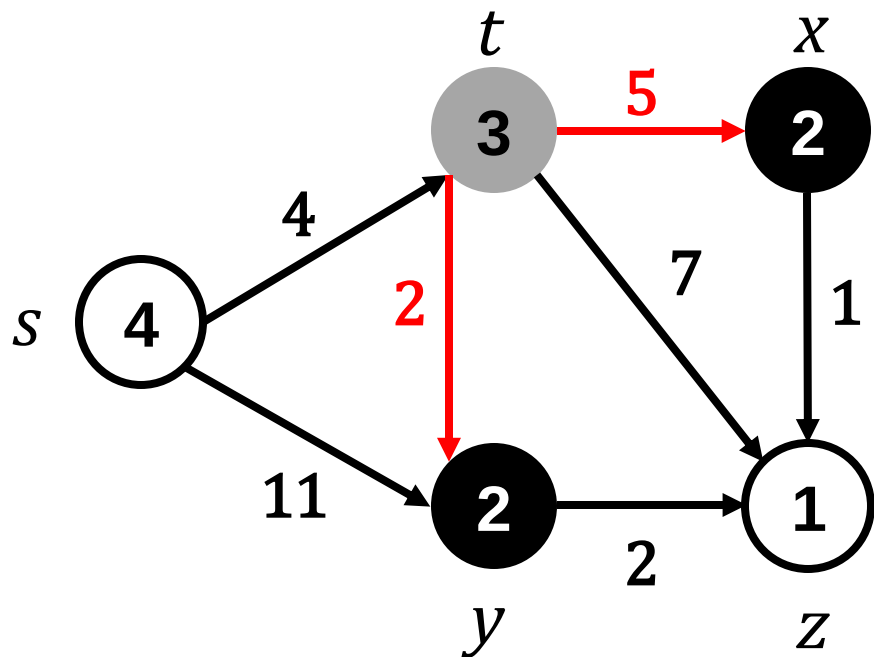
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0			4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

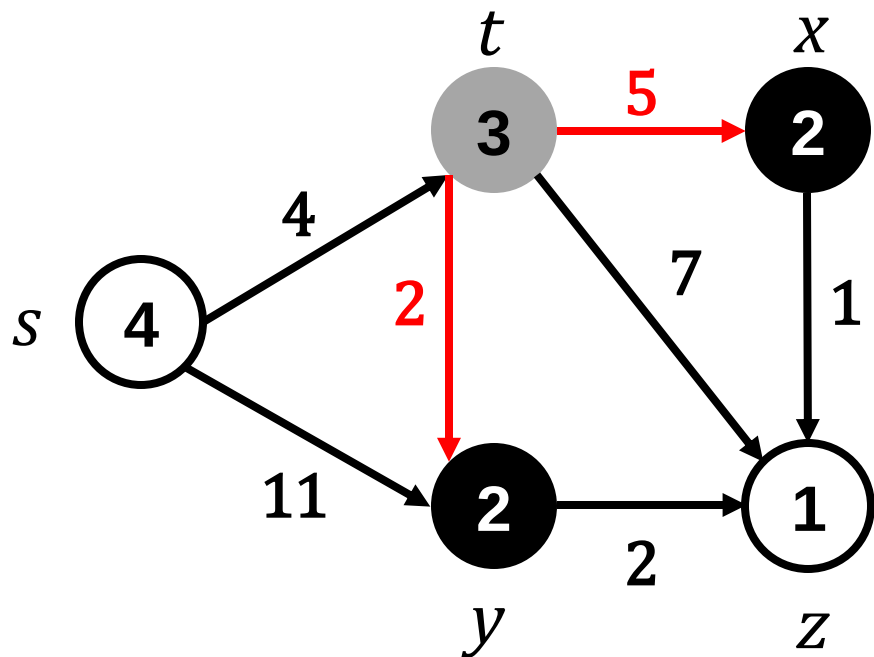
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



$dist$	s	t	x	y	z
s	0				
t	∞	0			4
x	∞	∞	0	∞	1
y	∞	∞	∞	0	2
z	∞	∞	∞	∞	0

问题实例

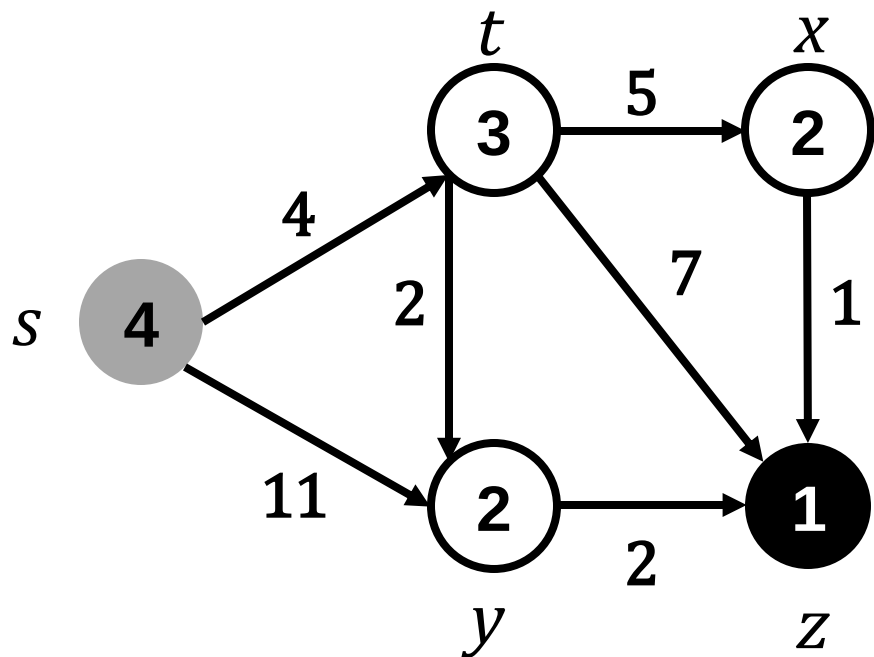
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



$dist$	s	t	x	y	z
s	0				
t	∞	0	5	2	4
x	∞	∞	0	∞	1
y	∞	∞	∞	0	2
z	∞	∞	∞	∞	0

问题实例

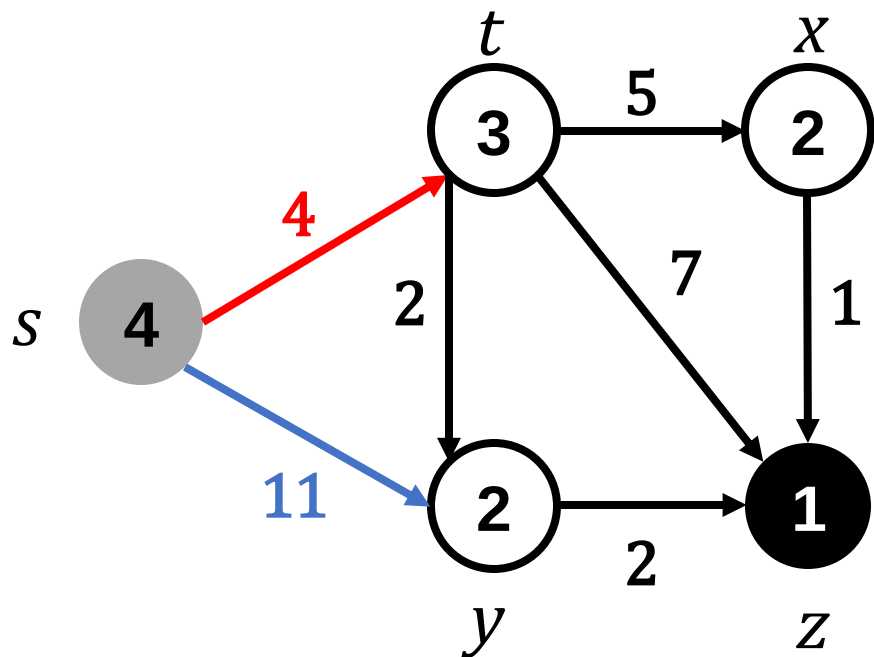
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

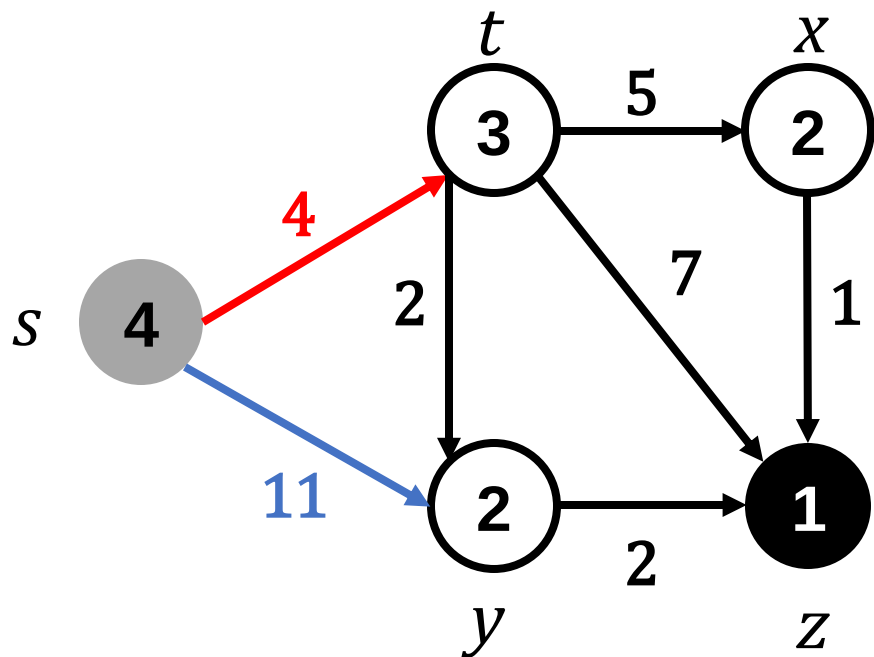
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

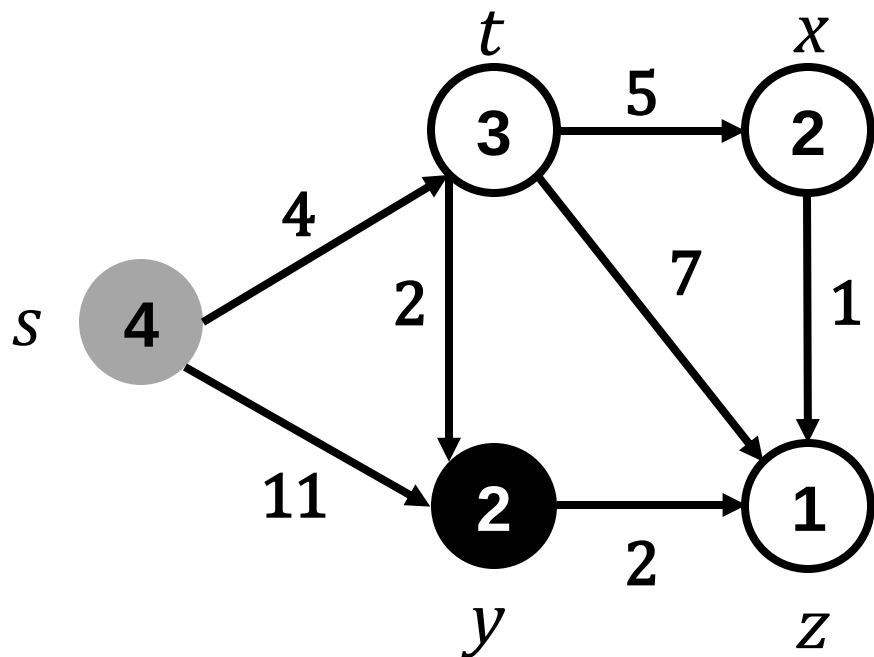
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

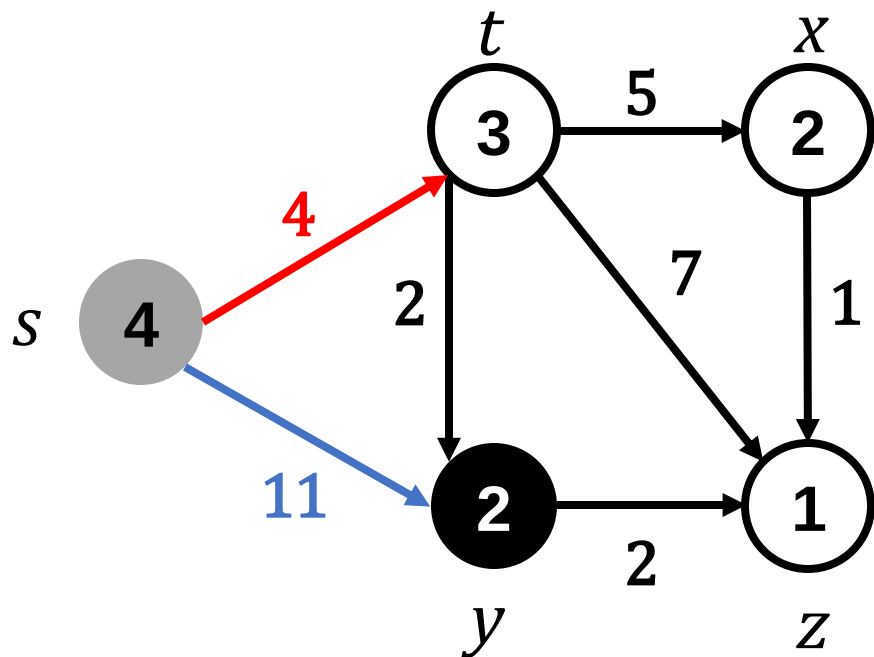
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

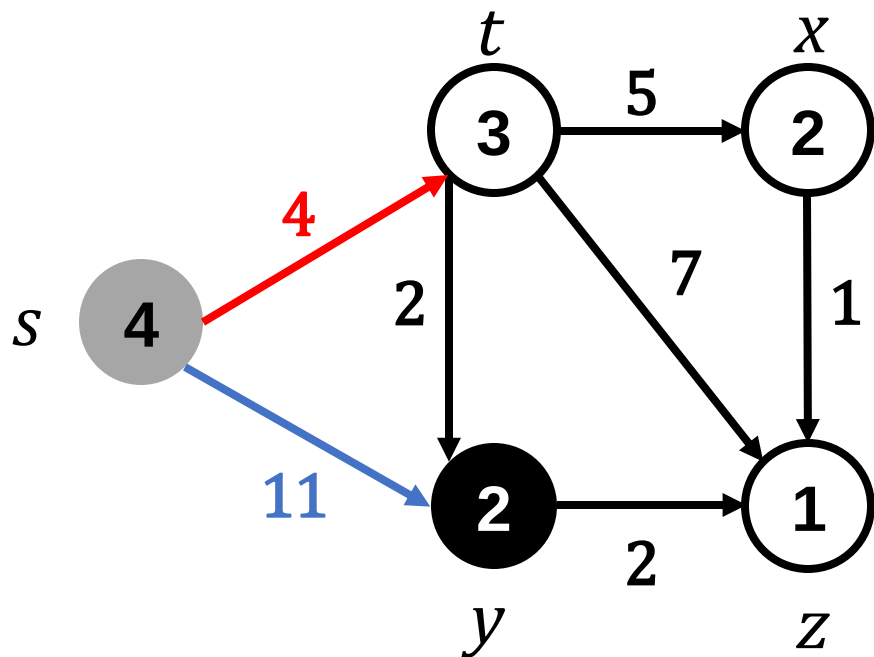
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0				8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

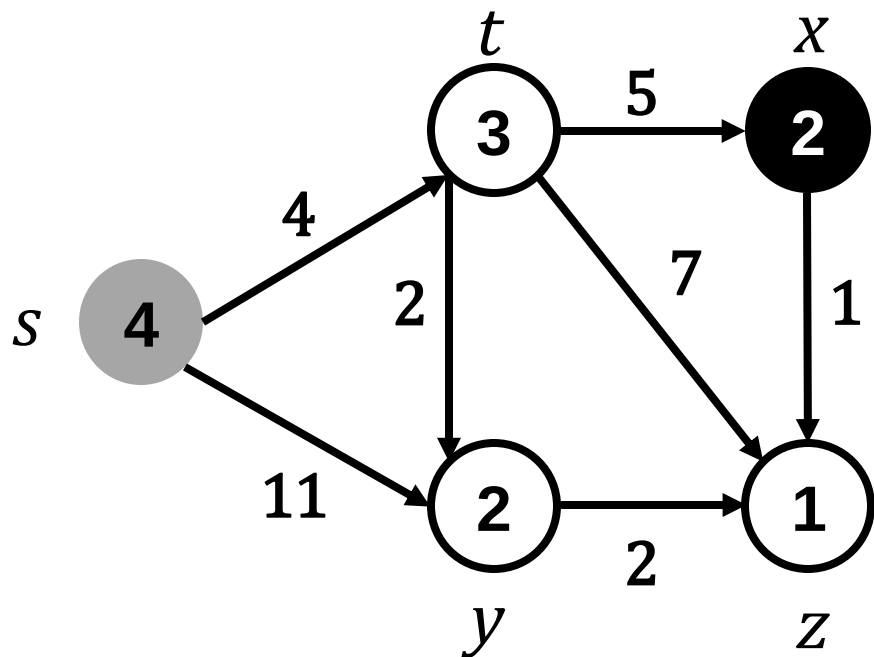
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0			6	8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

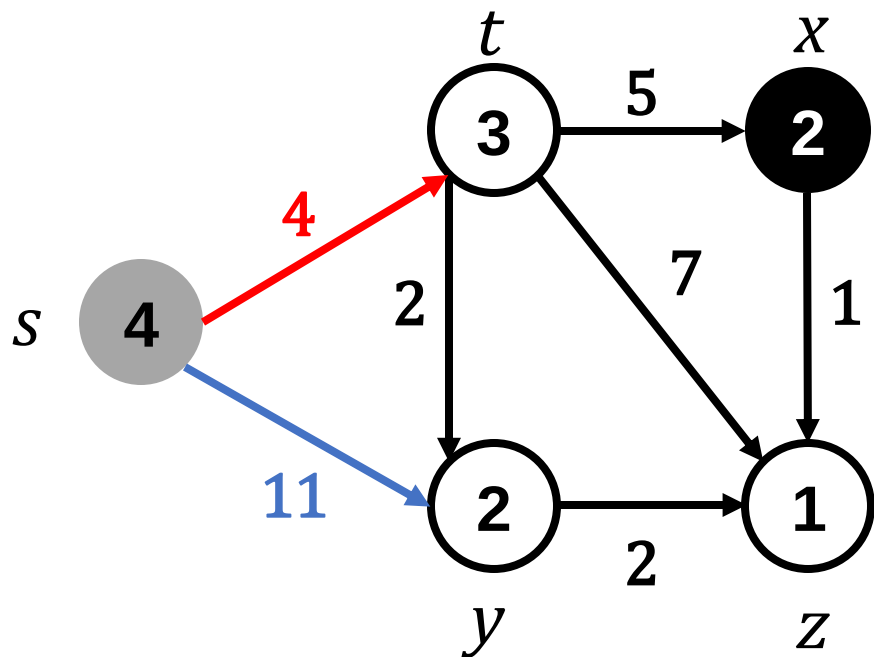
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0			6	8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

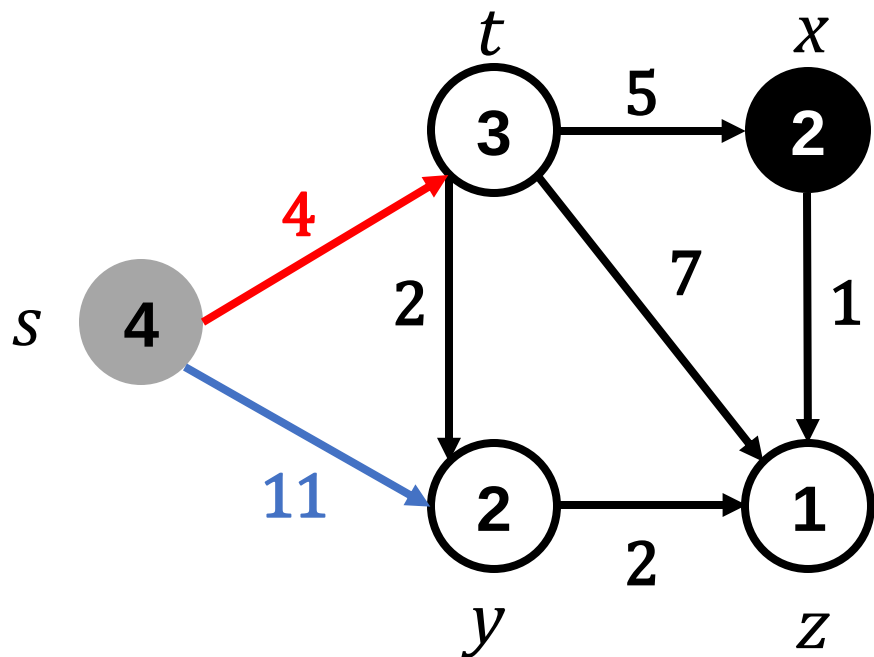
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0			6	8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

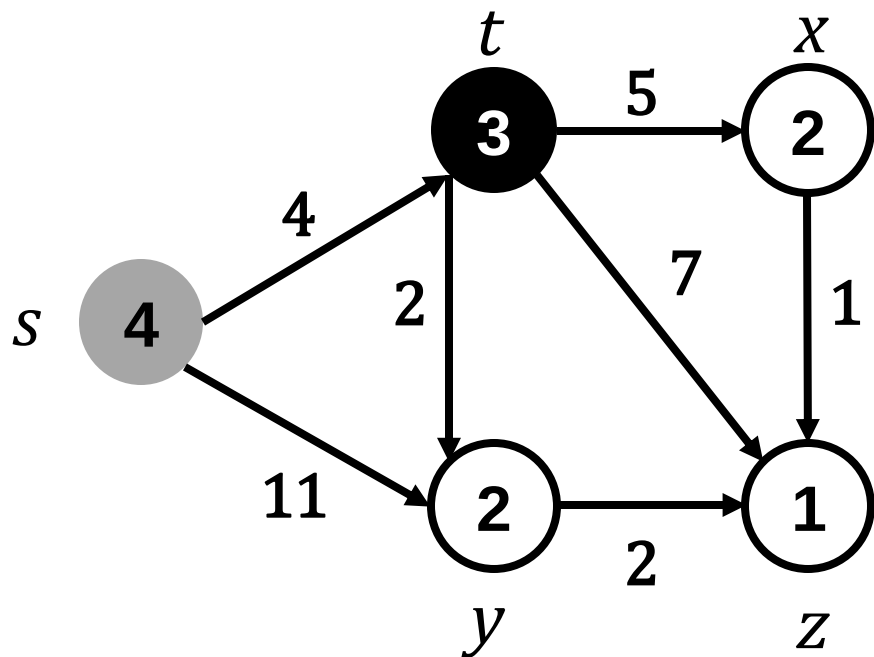
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0		9	6	8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

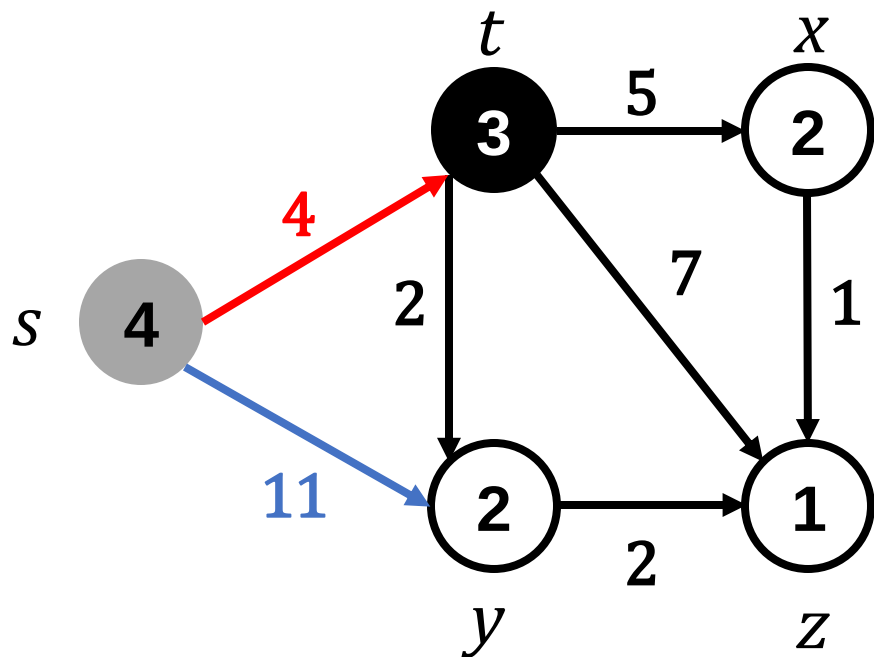
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0		9	6	8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

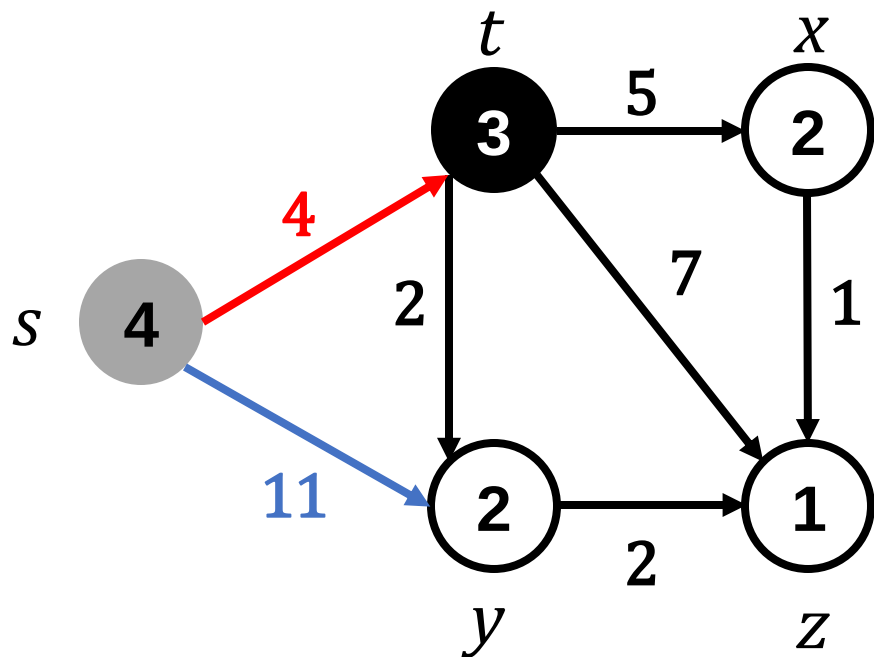
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0		9	6	8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

问题实例

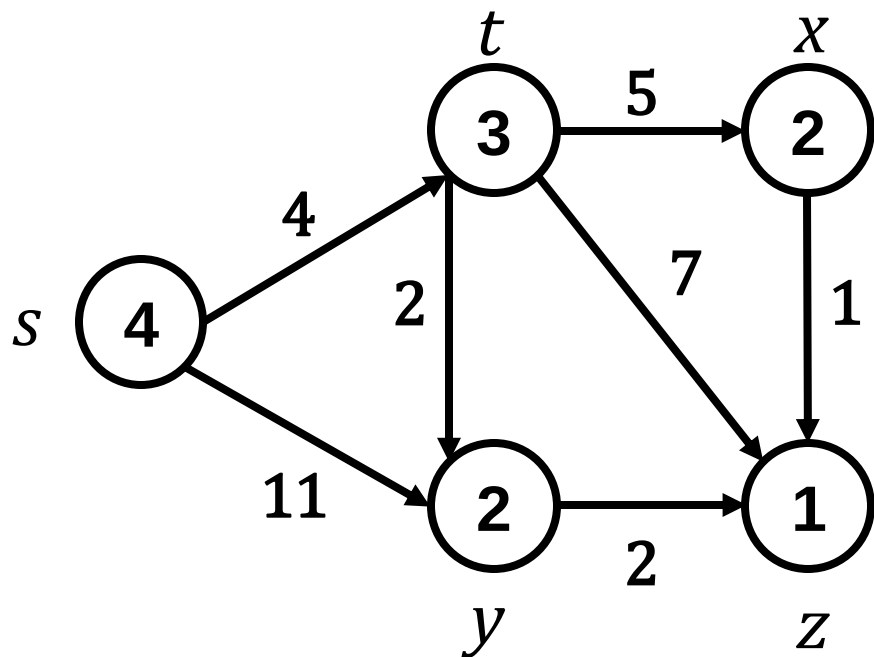
- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0	4	9	6	8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

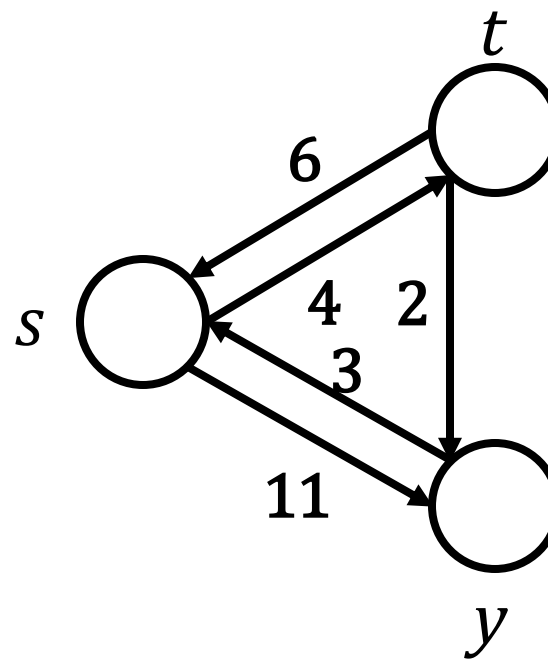
问题实例

- 递推公式: $dist[u, v] = \min_{(u, v_i) \in E} \{W(u, v_i) + dist[v_i, v]\}$



<i>dist</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>s</i>	0	4	9	6	8
<i>t</i>	∞	0	5	2	4
<i>x</i>	∞	∞	0	∞	1
<i>y</i>	∞	∞	∞	0	2
<i>z</i>	∞	∞	∞	∞	0

- 如果不是有向无环图 (DAG)
 - 图中存在环路



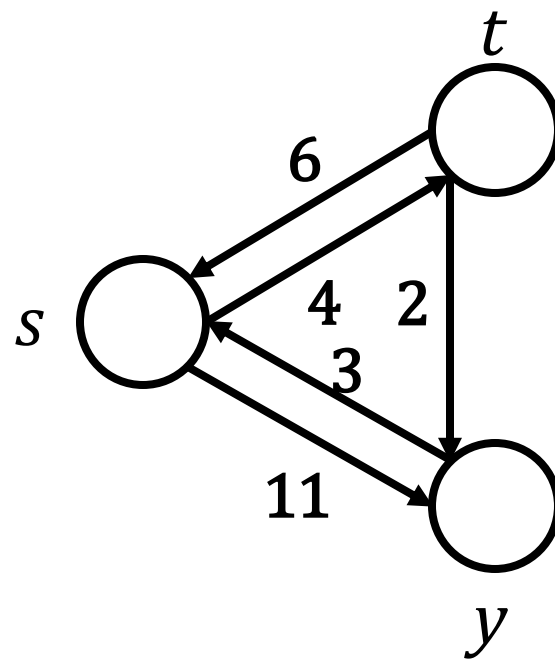
问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- $$dist[s, t] = \min \begin{cases} W(s, t) + dist[t, t] \\ W(s, y) + dist[y, t] \end{cases}$$



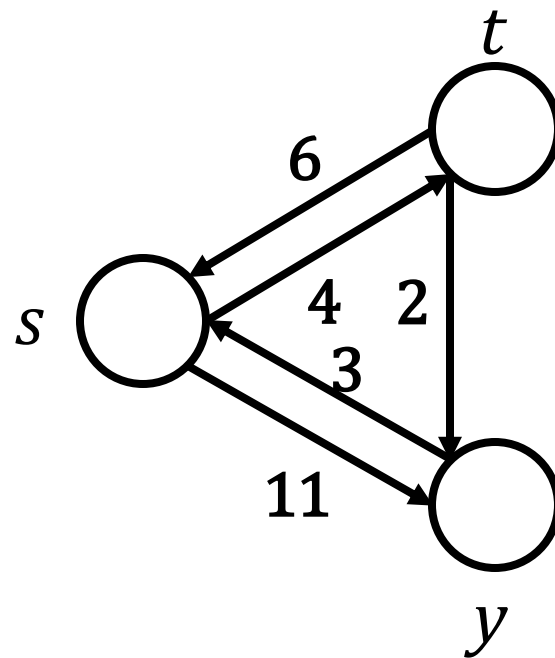
问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- $dist[s, t] = \min \begin{cases} W(s, t) + dist[t, t] \\ W(s, y) + \textcolor{red}{dist}[y, t] \end{cases}$
- $\textcolor{red}{dist}[y, t] = \min \begin{cases} W(y, t) + dist[t, t] \\ W(y, s) + dist[s, t] \end{cases}$



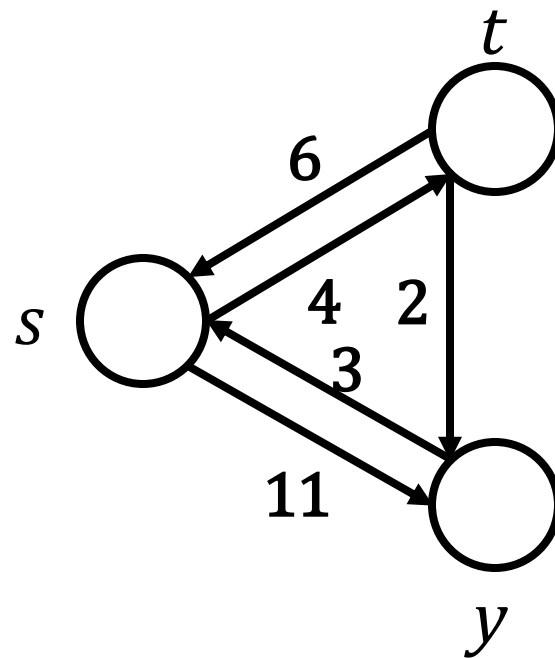
问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- $dist[s, t] = \min \begin{cases} W(s, t) + dist[t, t] \\ W(s, y) + \textcolor{red}{dist}[y, t] \end{cases}$
 - $\textcolor{red}{dist}[y, t] = \min \begin{cases} W(y, t) + dist[t, t] \\ W(y, s) + \textcolor{blue}{dist}[s, t] \end{cases}$
- 反向依赖，有后效性！



问题结构分析

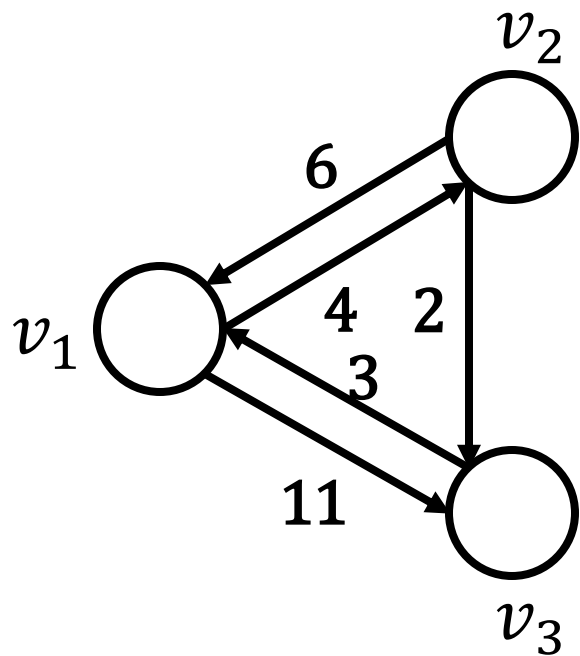
递推关系建立

自底向上计算

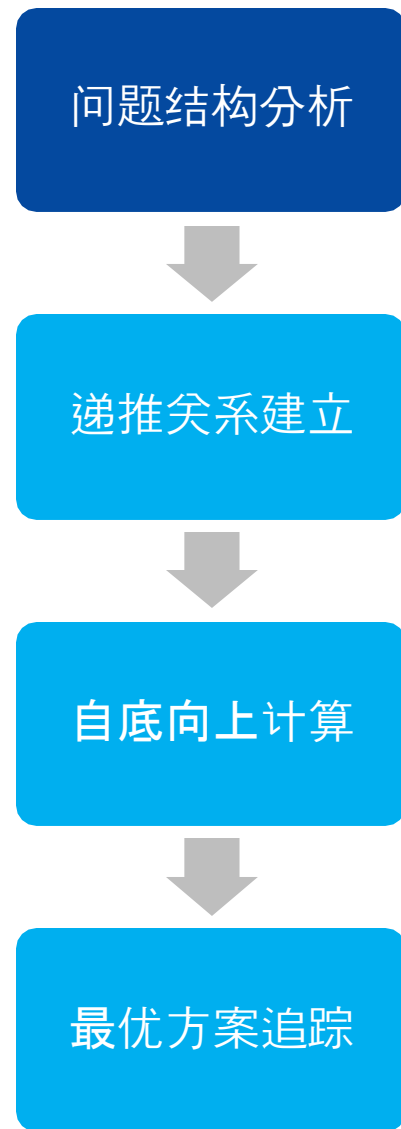
最优方案追踪

问题：如何消除后效性？

- 输入：
 - 带权图 $G = \langle V = \{v_1, v_2, \dots, v_n\}, E, W \rangle$, 对于所有的 $(v_i, v_j) \in E$, $W(v_i, v_j) \geq 0$
- 输出：
 - 从图中任意顶点 v_i 到 v_j 的最短距离



- 把最短路径问题看作是一个多阶段决策问题
 - 每步决策选择路径经过的**序号最大顶点**
- 令 $dist^k[i, j]$ 为顶点 v_i 到 v_j 只经过 $v_1, v_2, \dots, v_k$ 的最短路径
 - 原问题求解所有 $dist^n[i, j], \forall i, j = 1, 2, \dots, n$



- 令 $dist^k[i, j]$ 为顶点 v_i 到 v_j 只经过 $v_1, v_2, \dots, v_k$ 的最短路径
 - 原问题求解所有 $dist^n[i, j], \forall i, j = 1, 2, \dots, n$
- 若 $v_i \rightarrow \dots \rightarrow v_k \rightarrow \dots \rightarrow v_j$ 是顶点 v_i 到 v_j 只经过 $v_1, v_2, \dots, v_k$ 的最短路径
 - 那么 $v_i \rightarrow \dots \rightarrow v_k$ 一定是 v_i 到 v_k 只经过 $v_1, v_2, \dots, v_{k-1}$ 的最短路径
 - $v_k \rightarrow \dots \rightarrow v_j$ 一定是 v_k 到 v_j 只经过 $v_1, v_2, \dots, v_{k-1}$ 的最短路径
- 满足最优子结构性质
- 递推公式:

$$dist^k[i, j] = \min\{dist^{k-1}[i, k] + dist^{k-1}[k, j], dist^{k-1}[i, j]\}$$

问题：如何确定计算顺序？

思考：为什么？

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 对于任意 i 和 $j = 1, 2, \dots, n$: $dist^0[i, j] = W(i, j)$
- 对于 $k = 1, 2, \dots, n$, 计算 $dist^k$:

$$dist^k[i, j] = \min\{dist^{k-1}[i, k] + dist^{k-1}[k, j], dist^{k-1}[i, j]\}$$

- 返回最终结果 $dist^n$

问题结构分析



递推关系建立

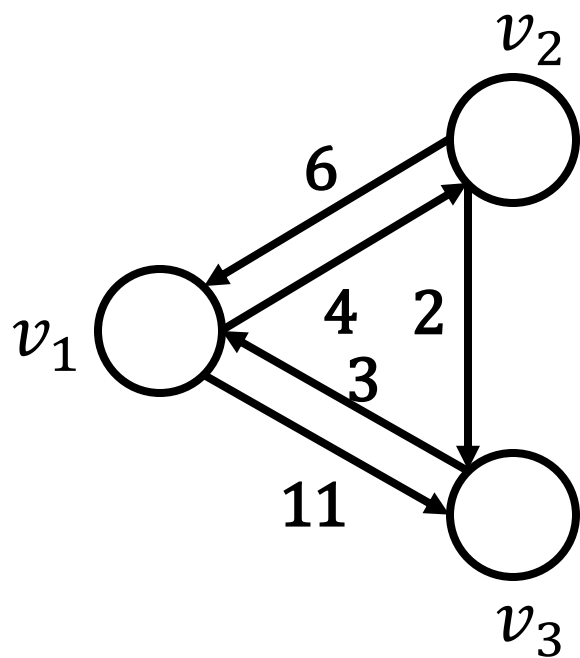


自底向上计算

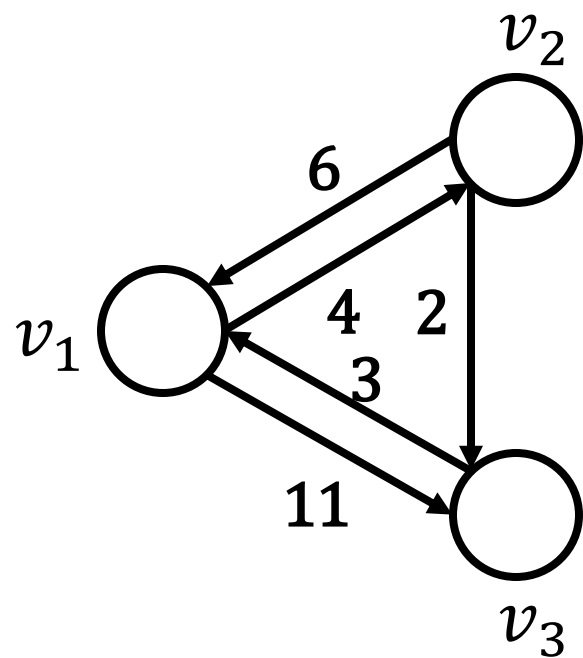


最优方案追踪

思考：如何需要输出最短路径，如何追踪最优方案？

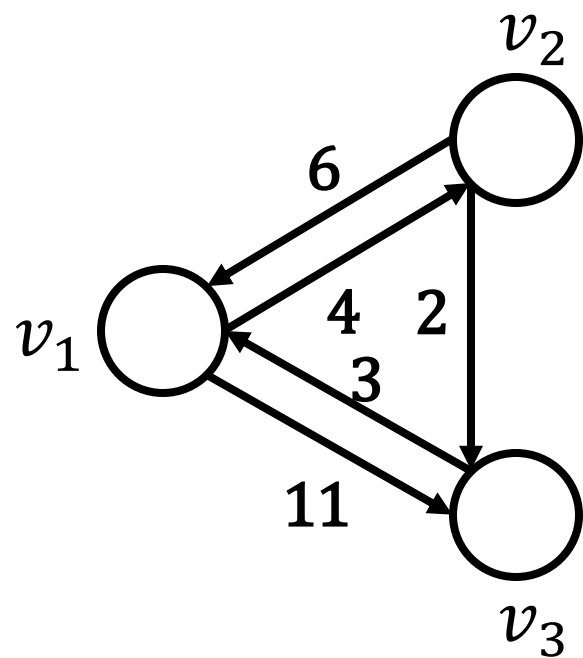


$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

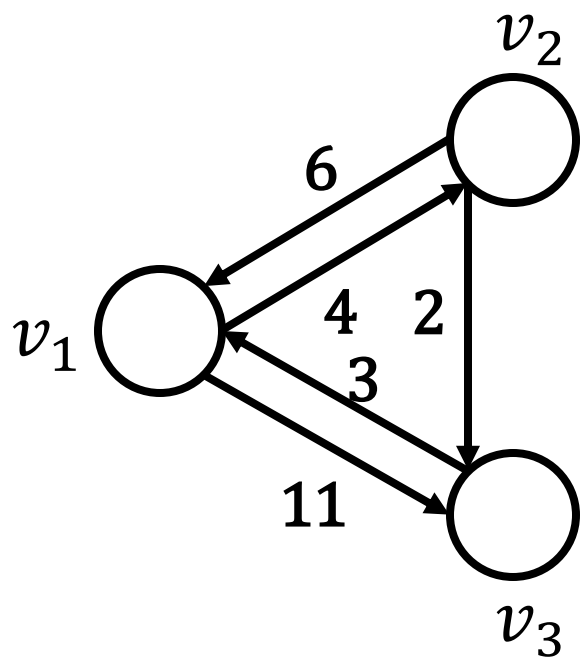
$dist^1$	v_1	v_2	v_3
v_1	0		
v_2		0	
v_3			0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

$$dist^1[1, j] = dist^0[1, j]$$

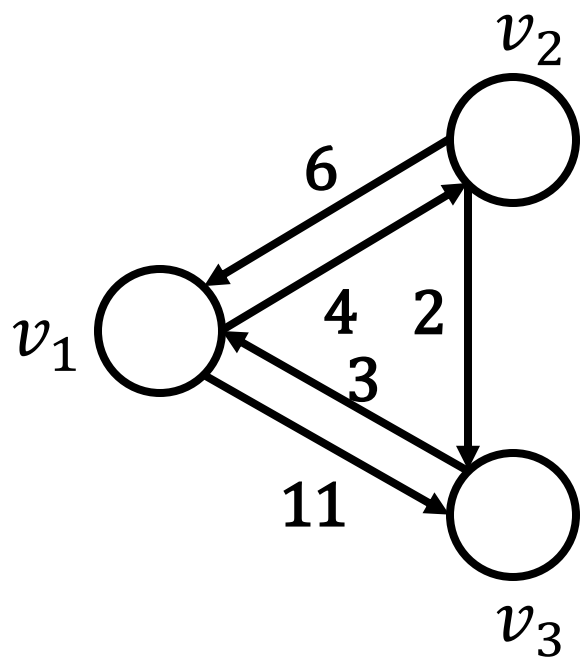
$dist^1$	v_1	v_2	v_3
v_1	0		
v_2		0	
v_3			0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

$$dist^1[1, j] = dist^0[1, j]$$

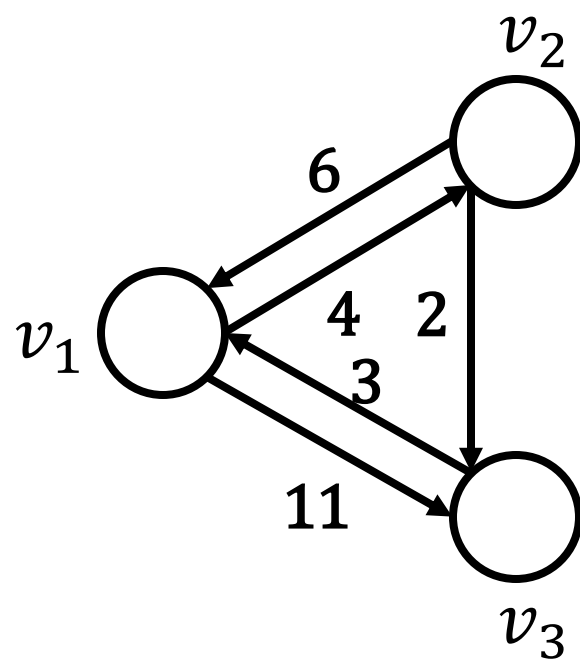
$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2		0	
v_3			0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

$$dist^1[i, 1] = dist^0[i, 1]$$

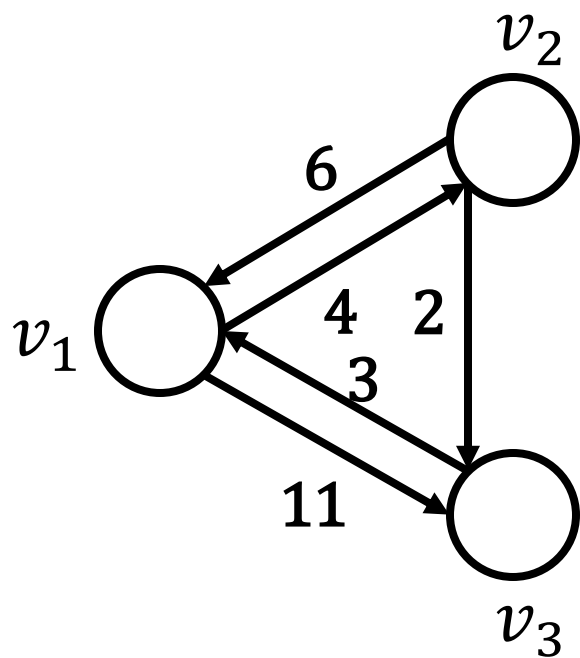
$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2		0	
v_3			0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

$$dist^1[i, 1] = dist^0[i, 1]$$

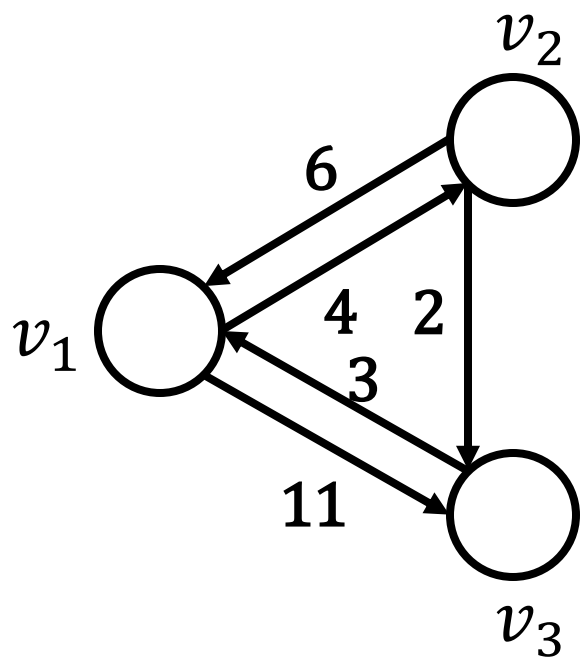
$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	
v_3	3		0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

$$dist^1[2,3] = \min\{dist^0[2,1] + dist^0[1,3], dist^0[2,3]\}$$

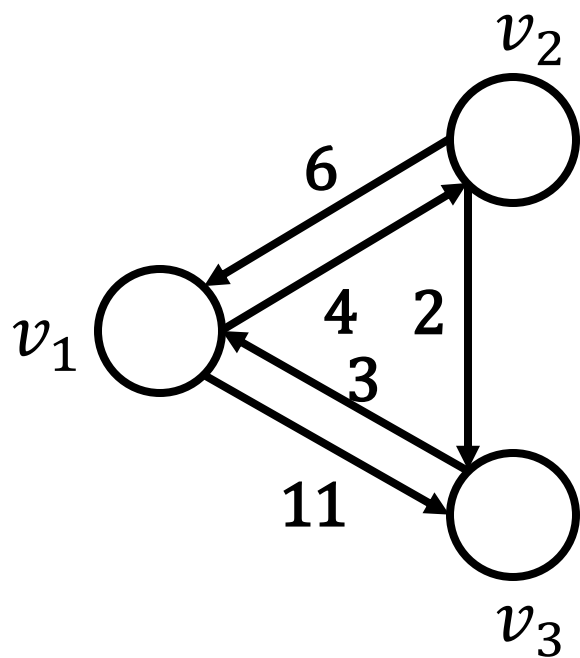
$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	
v_3	3		0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

$$dist^1[2,3] = \min\{dist^0[2,1] + dist^0[1,3], dist^0[2,3]\}$$

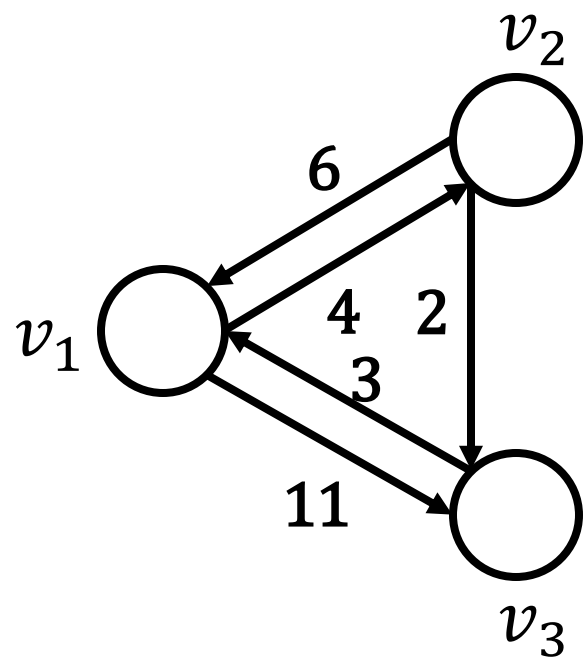
$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3		0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

$$dist^1[3,2] = \min\{dist^0[3,1] + dist^0[1,2], dist^0[3,2]\}$$

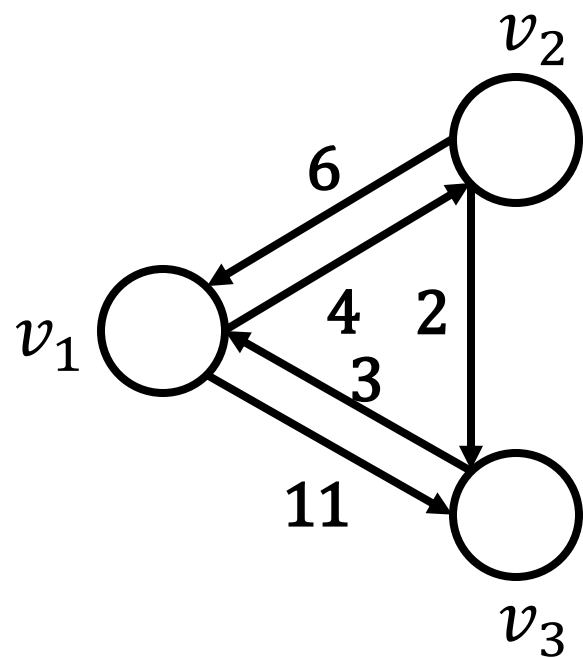
$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3		0



$dist^0$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	∞	0

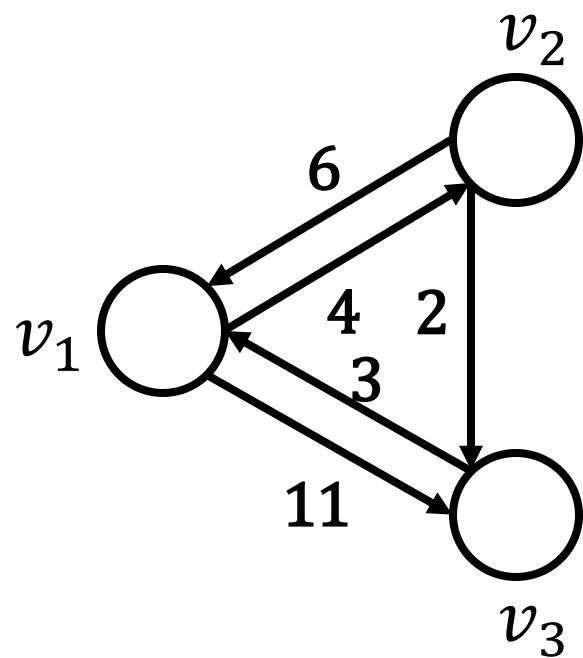
$$dist^1[3,2] = \min\{dist^0[3,1] + dist^0[1,2], dist^0[3,2]\}$$

$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

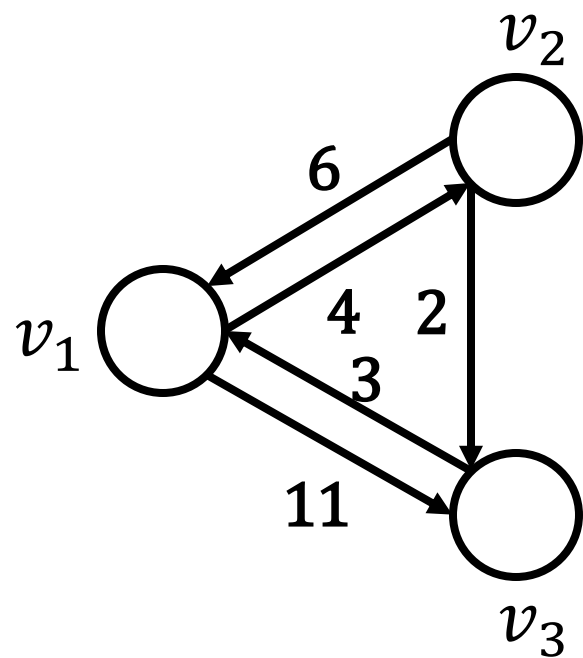
$dist^2$	v_1	v_2	v_3
v_1	0		
v_2		0	
v_3			0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

$$dist^2[2, j] = dist^1[2, j]$$

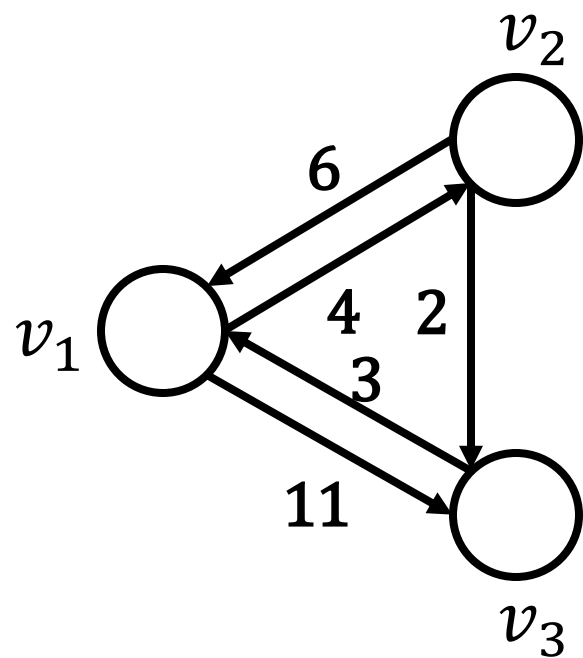
$dist^2$	v_1	v_2	v_3
v_1	0		
v_2		0	
v_3			0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

$$dist^2[2, j] = dist^1[2, j]$$

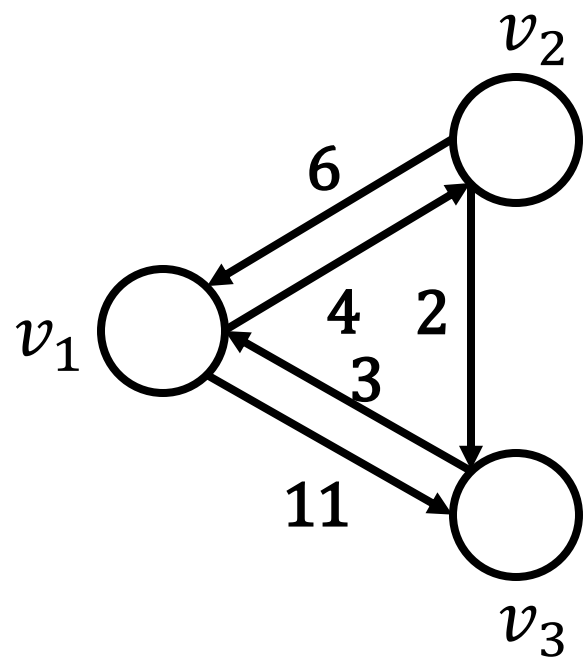
$dist^2$	v_1	v_2	v_3
v_1	0		
v_2	6	0	2
v_3			0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

$$dist^2[i, 2] = dist^1[i, 2]$$

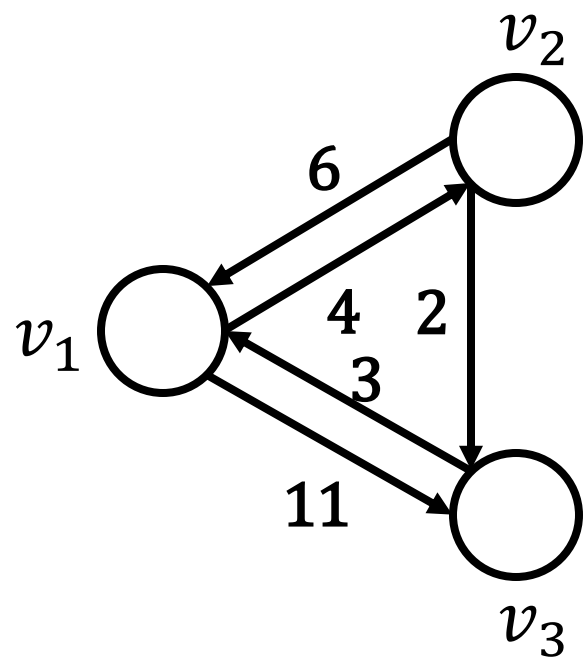
$dist^2$	v_1	v_2	v_3
v_1	0		
v_2	6	0	2
v_3			0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

$$dist^2[i, 2] = dist^1[i, 2]$$

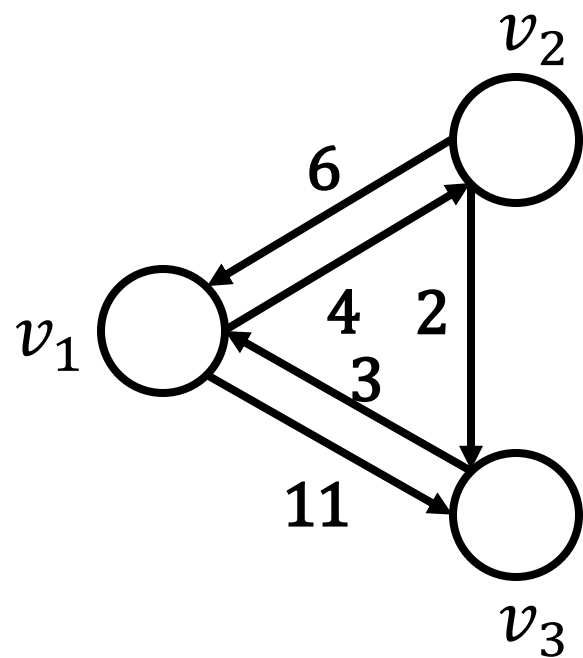
$dist^2$	v_1	v_2	v_3
v_1	0	4	
v_2	6	0	2
v_3		7	0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

$$dist^2[1,3] = \min\{dist^1[1,2] + dist^1[2,3], dist^1[1,3]\}$$

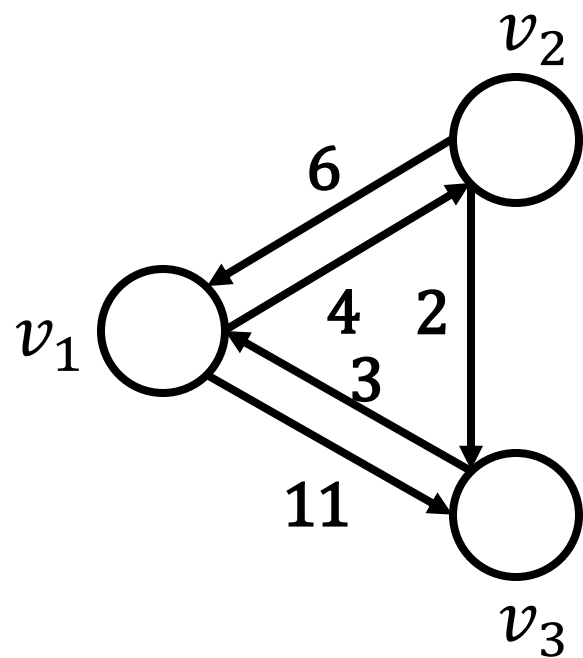
$dist^2$	v_1	v_2	v_3
v_1	0	4	
v_2	6	0	2
v_3		7	0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

$$dist^2[1,3] = \min\{dist^1[1,2] + dist^1[2,3], dist^1[1,3]\}$$

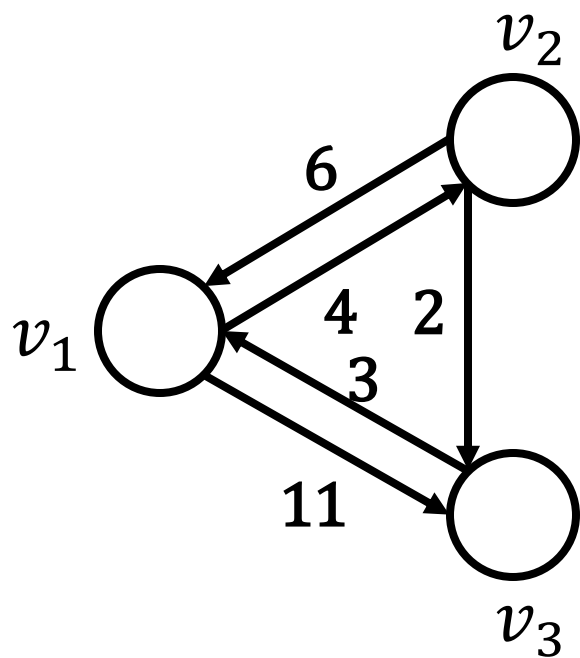
$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3		7	0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

$$dist^2[3,1] = \min\{dist^1[3,2] + dist^1[2,1], dist^1[3,1]\}$$

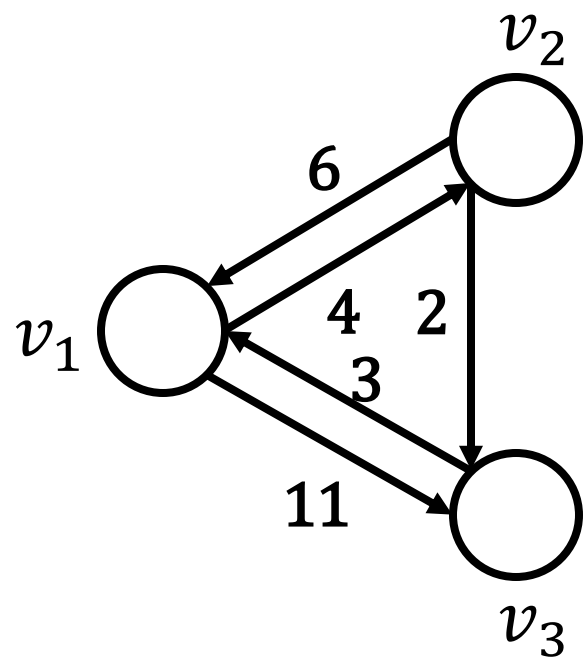
$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3		7	0



$dist^1$	v_1	v_2	v_3
v_1	0	4	11
v_2	6	0	2
v_3	3	7	0

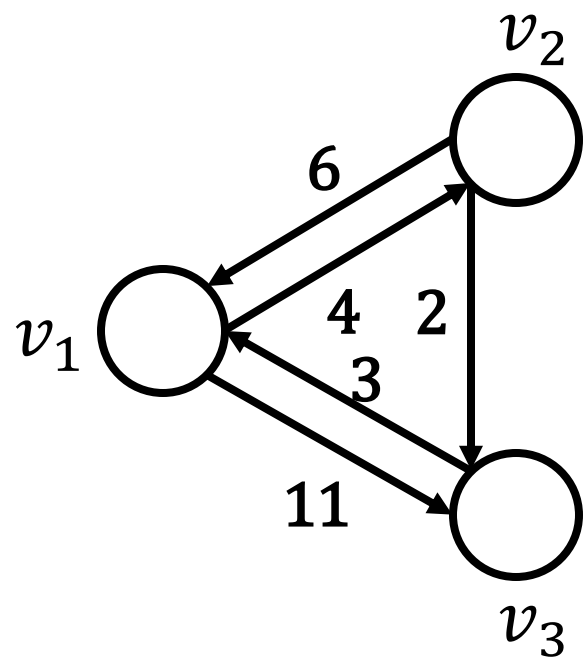
$$dist^2[3,1] = \min\{dist^1[3,2] + dist^1[2,1], dist^1[3,1]\}$$

$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

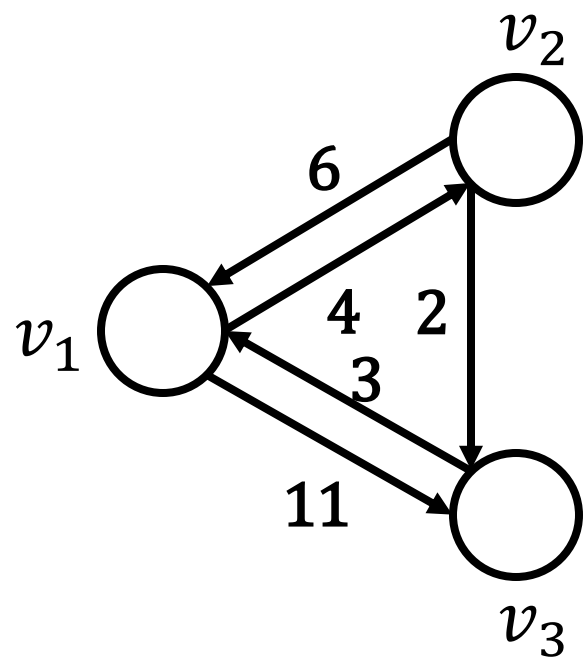
$dist^3$	v_1	v_2	v_3
v_1	0		
v_2		0	
v_3			0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

$$dist^3[3, j] = dist^2[3, j]$$

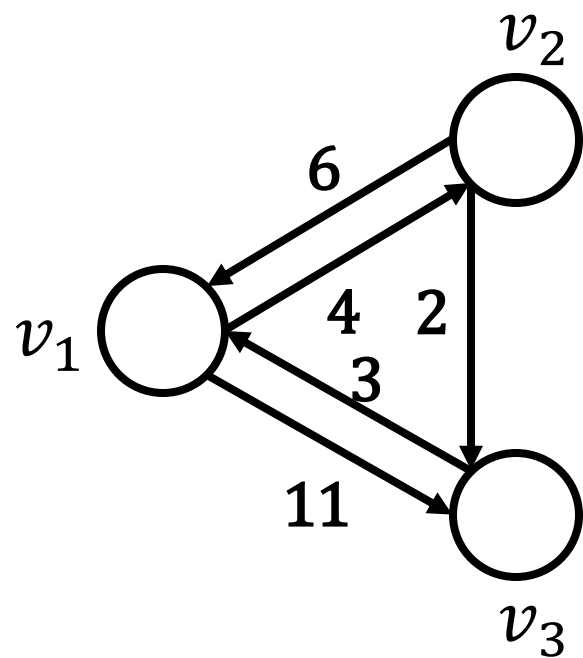
$dist^3$	v_1	v_2	v_3
v_1	0		
v_2		0	
v_3			0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

$$dist^3[3, j] = dist^2[3, j]$$

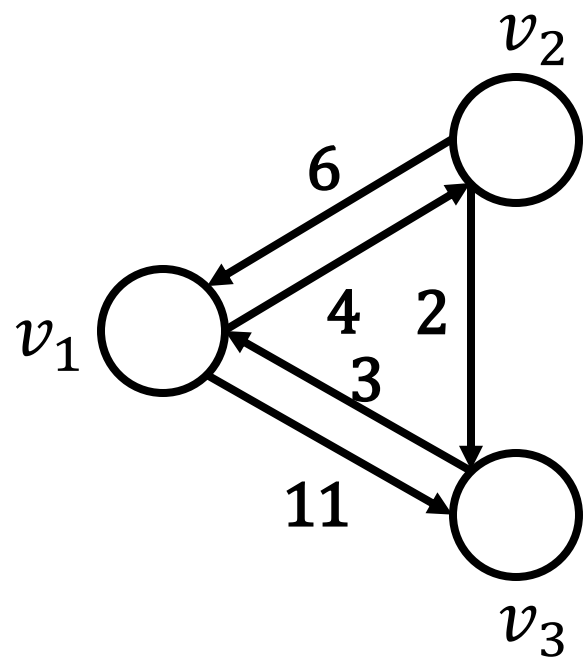
$dist^3$	v_1	v_2	v_3
v_1	0		
v_2		0	
v_3	3	7	0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

$$dist^3[i, 3] = dist^2[i, 3]$$

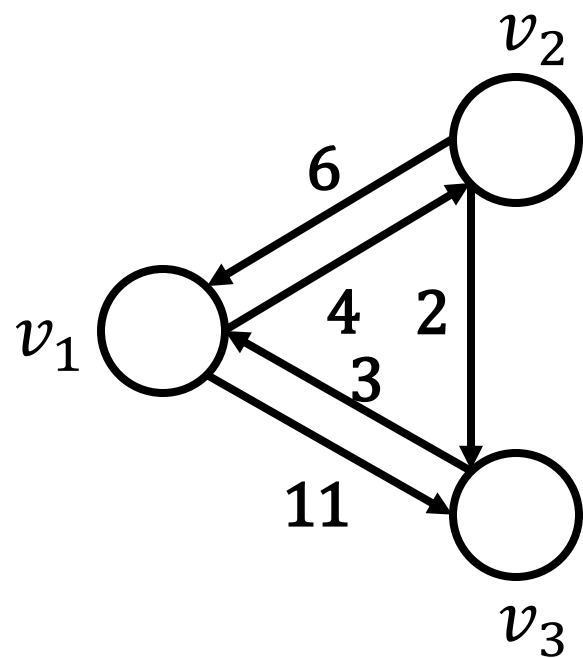
$dist^3$	v_1	v_2	v_3
v_1	0		
v_2		0	
v_3	3	7	0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

$$dist^3[i, 3] = dist^2[i, 3]$$

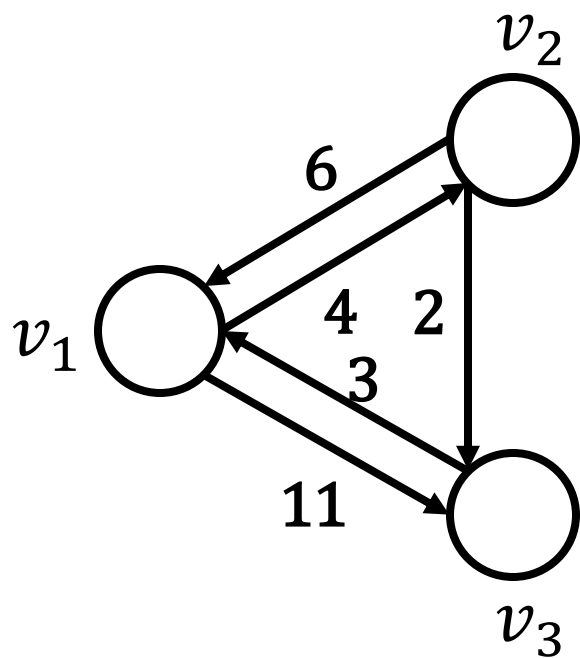
$dist^3$	v_1	v_2	v_3
v_1	0		6
v_2		0	2
v_3	3	7	0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

$$dist^3[1,2] = \min\{dist^2[1,3] + dist^2[3,2], dist^2[1,2]\}$$

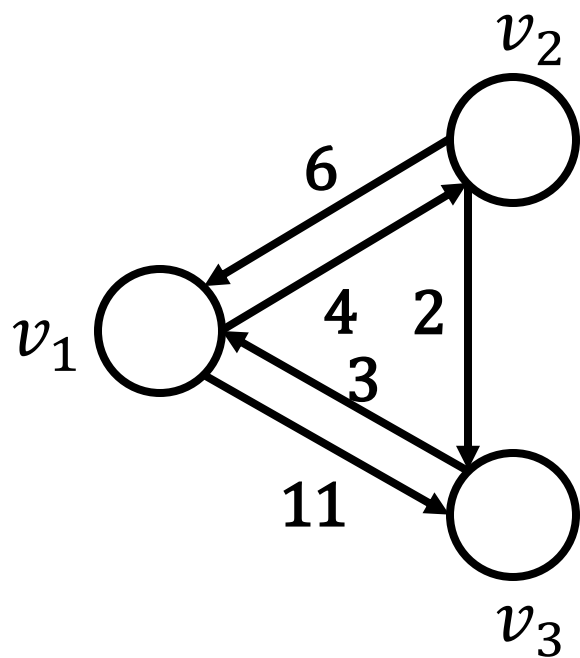
$dist^3$	v_1	v_2	v_3
v_1	0		6
v_2		0	2
v_3	3	7	0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

$$dist^3[1,2] = \min\{dist^2[1,3] + dist^2[3,2], dist^2[1,2]\}$$

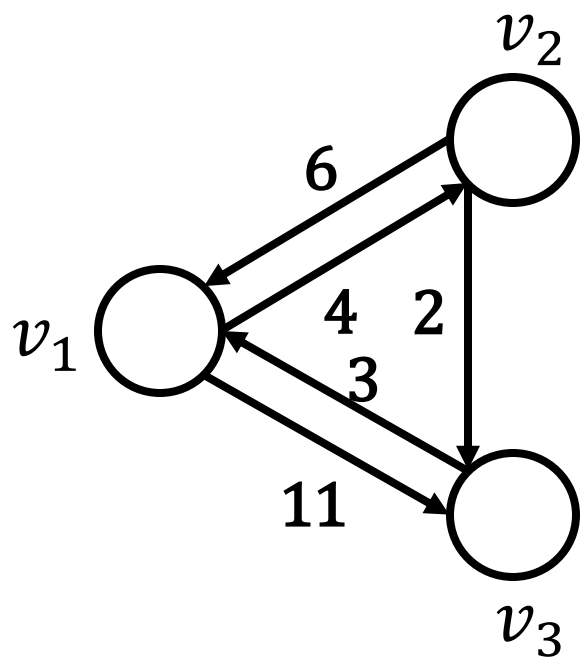
$dist^3$	v_1	v_2	v_3
v_1	0	4	6
v_2		0	2
v_3	3	7	0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

$$dist^3[2,1] = \min\{dist^2[2,3] + dist^2[3,1], dist^2[2,1]\}$$

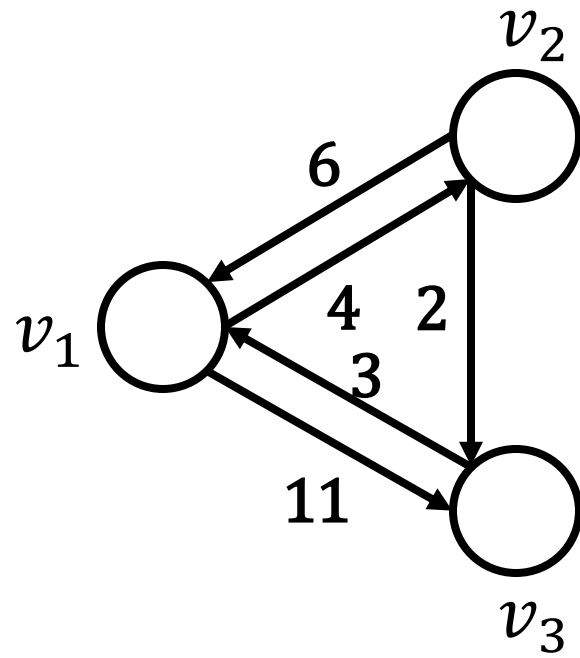
$dist^3$	v_1	v_2	v_3
v_1	0	4	6
v_2		0	2
v_3	3	7	0



$dist^2$	v_1	v_2	v_3
v_1	0	4	6
v_2	6	0	2
v_3	3	7	0

$$dist^3[2,1] = \min\{dist^2[2,3] + dist^2[3,1], dist^2[2,1]\}$$

$dist^3$	v_1	v_2	v_3
v_1	0	4	6
v_2	5	0	2
v_3	3	7	0



$dist$	v_1	v_2	v_3
v_1	0	4	6
v_2	5	0	2
v_3	3	7	0

输入：带权图 $G = \langle V = \{v_1, v_2, \dots, v_n\}, E, W \rangle$ ，对于所有的 $(v_i, v_j) \in E$ ， $W(v_i, v_j) \geq 0$

输出：二维数组 A ， $A[v_i, v_j]$ 表示 v_i 到 v_j 的最短路径

```
for i ← 0 to n do
    for j ← 0 to n do
         $A[i, j] \leftarrow W(v_i, v_j)$ 
    end
end
for k ← 1 to n do
    for i ← 0 to n do
        for j ← 0 to n do
             $A[i, j] = \min\{A[i, j], A[i, k] + A[k, j]\}$ 
        end
    end
end
```

输入：带权图 $G = \langle V = \{v_1, v_2, \dots, v_n\}, E, W \rangle$ ，对于所有的 $(v_i, v_j) \in E$ ， $W(v_i, v_j) \geq 0$

输出：二维数组 A ， $A[v_i, v_j]$ 表示 v_i 到 v_j 的最短路径

```
for i ← 0 to n do
  for j ← 0 to n do
     $A[i, j] \leftarrow W(v_i, v_j)$ 
  end
end
```

初始化

```
for k ← 1 to n do
  for i ← 0 to n do
    for j ← 0 to n do
       $A[i, j] = \min\{A[i, j], A[i, k] + A[k, j]\}$ 
    end
  end
end
```

输入：带权图 $G = \langle V = \{v_1, v_2, \dots, v_n\}, E, W \rangle$ ，对于所有的 $(v_i, v_j) \in E$ ， $W(v_i, v_j) \geq 0$

输出：二维数组 A ， $A[v_i, v_j]$ 表示 v_i 到 v_j 的最短路径

for $i \leftarrow 0$ to n do

 for $j \leftarrow 0$ to n do

$A[i, j] \leftarrow W(v_i, v_j)$

 end

end

for $k \leftarrow 1$ to n do

 for $i \leftarrow 0$ to n do

 for $j \leftarrow 0$ to n do

$A[i, j] = \min\{A[i, j], A[i, k] + A[k, j]\}$

 end

 end

end

迭代计算

输入：带权图 $G = \langle V = \{v_1, v_2, \dots, v_n\}, E, W \rangle$ ，对于所有的 $(v_i, v_j) \in E$ ， $W(v_i, v_j) \geq 0$

输出：二维数组 A ， $A[v_i, v_j]$ 表示 v_i 到 v_j 的最短路径

```
for i ← 0 to n do
    for j ← 0 to n do
         $A[i, j] \leftarrow W(v_i, v_j)$ 
    end
end
for k ← 1 to n do
    for i ← 0 to n do
        for j ← 0 to n do
             $A[i, j] = \min\{A[i, j], A[i, k] + A[k, j]\}$ 
        end
    end
end
end
```

时间复杂度： $O(n^3)$

- 又称Floyd算法，Robert W. Floyd于1962年发明
- 其他最短路径算法：
 - Bellman-Ford算法
 - Dijkstra算法

	Floyd算法	Bellman–Ford算法	Dijkstra算法
最短路类型	每对结点最短路	单源最短路	单源最短路
适用范围	任意图	任意图	非负权图
算法思想	动态规划	动态规划	贪心（动态规划）
时间复杂度	$O(V ^3)$	$O(V \cdot E)$	$O(E \log V)$

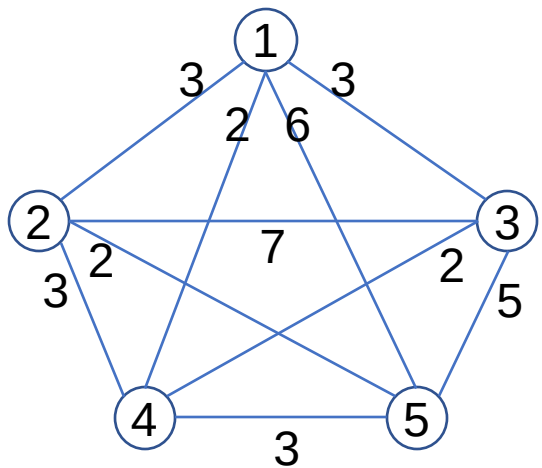


Robert W. Floyd
1936 – 2001
1978图灵奖得主

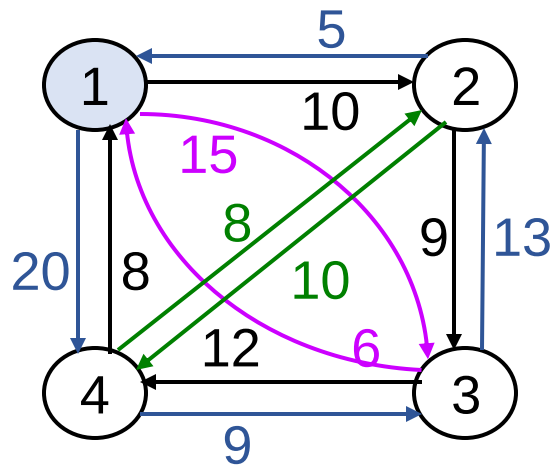
- 5.1 0-1背包问题
- 5.2 动态规划主要思想
- 5.3 再论背包问题
- 5.4 可靠性设计
- 5.5 最短路径问题
- **5.6 再论货郎担问题**
- 5.7 最长公共子序列
- 5.8 编辑距离问题
- 5.9 最优二分检索树
- 5.10 矩阵链乘积问题

- 问题描述：某售货员要到若干个城市销售货物，已知各城市之间的距离，要求售货员从某一城市出发并选择旅行路线，使每个城市经过一次，最后回到原出发城市，而总路程最短
- 实际应用
 - 收集邮件的邮路问题
 - 机械手臂在装配线上的移动问题
 - 产品的生产安排问题

- 根据目标函数，选择距离当前城市最短距离的城市
 - 从某一个城市开始，每次选择一个新城市，直到所有的城市被走完
 - 在选择下一个城市时，只考虑当前情况，保证当前选择的城市距离最小且不形成回路



	1	2	3	4	5
1	∞	3	3	2	6
2	3	∞	7	3	2
3	3	7	∞	2	5
4	2	3	2	∞	3
5	6	2	5	3	∞



设 $G(V,E)$ 是一个具有边成本 c_{ij} 的有向图, G 的一条周游路线是包含 V 中每个结点的一个有向环, 周游路线的成本是此路线上所有边的成本之和

- 假设周游路线是开始于结点1并终止于结点1的一条简单路径
 - 每一条周游路线都是由一条边 $(1, k)$ 和一条由结点 k 到结点1的简单路径所组成的, 其中 $k \in V \setminus \{1\}$
 - 这条由结点 k 到结点1的路径通过 $k \in V \setminus \{1\}$ 的每个结点各一次
- 如果这条周游路线是最优的, 则这条由 k 到1的路径必定是通过 $k \in V \setminus \{1\}$ 中所有结点的由 k 到1的最短路径, 因此**最优性原理成立**

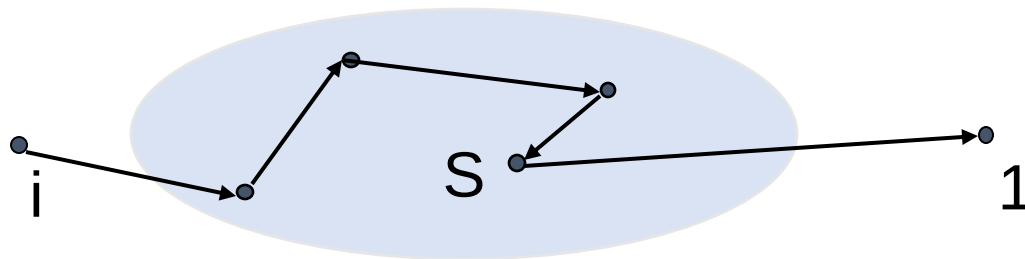
问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 设 $g(i, S)$ 是由结点 i 开始, 通过 S 中的所有结点, 在结点1终止的一条最短路径长度
 - $g(1, V \setminus \{1\})$ 是一条最优的周游路线长度
 - $g(1, V \setminus \{1\}) = \min_{2 \leq k \leq n} \{c_{1k} + g(k, V \setminus \{1, k\})\}$



问题结构分析

递推关系建立

自底向上计算

最优方案追踪

- 设 $g(i, S)$ 是由结点 i 开始, 通过 S 中的所有结点, 在结点1终止的一条最短路径长度
 - $g(1, V \setminus \{1\})$ 是一条最优的周游路线长度
 - $g(1, V \setminus \{1\}) = \min_{2 \leq k \leq n} \{c_{1k} + g(k, V \setminus \{1, k\})\}$
- 递推公式
 - $g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S \setminus \{j\})\}$
 - 初始值: $g(i, \emptyset) = c_{i1}, 1 < i \leq n$

问题结构分析



递推关系建立

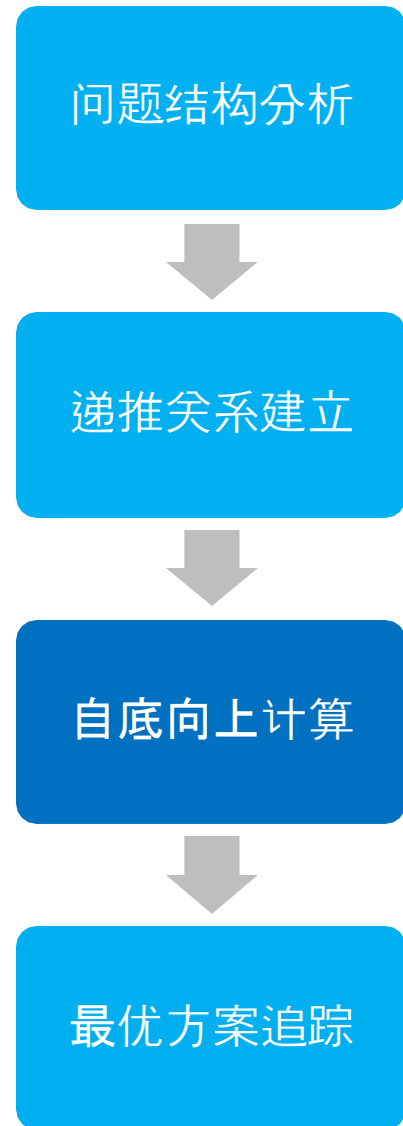


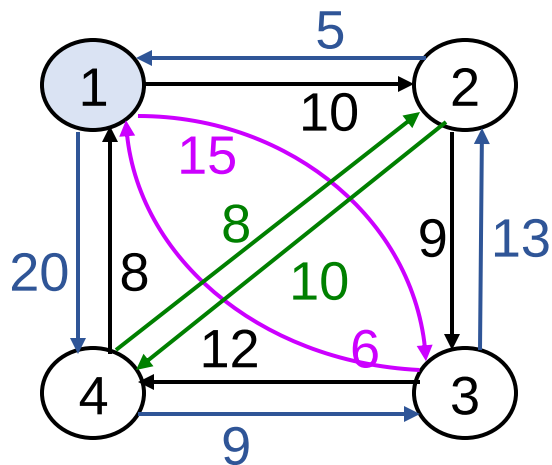
自底向上计算



最优方案追踪

- 设 $g(i, S)$ 是由结点 i 开始, 通过 S 中的所有结点, 在结点1终止的一条最短路径长度
 - $g(1, V \setminus \{1\})$ 是一条最优的周游路线长度
 - $g(1, V \setminus \{1\}) = \min_{2 \leq k \leq n} \{c_{1k} + g(k, V \setminus \{1, k\})\}$
- 递推公式
 - $g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S \setminus \{j\})\}$
 - 初始值: $g(i, \emptyset) = c_{i1}, 1 < i \leq n$
- 求解过程
 - 依次求出 $|S| = k$ 的所有 $g(i, S), k = 0, 1, \dots, n-1$
 - 当 $k < n-1$ 时, $i \neq 1, 1 \notin S$ 且 $i \notin S$
 - 当 $k = n-1$ 时, 求 $g(1, V \setminus \{1\})$, 即为最终解





	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

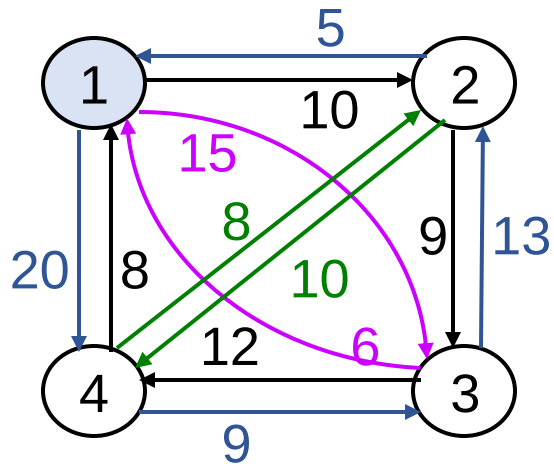
$|S|=0$

- $g(2, \emptyset) = c_{21} = 5$
- $g(3, \emptyset) = c_{31} = 6$
- $g(4, \emptyset) = c_{41} = 8$

$|S|=1$

- $g(2, \{3\}) = c_{23} + g(3, \emptyset) = 9 + 6 = 15$
- $g(2, \{4\}) = c_{24} + g(4, \emptyset) = 10 + 8 = 18$
- $g(3, \{2\}) = c_{32} + g(2, \emptyset) = 13 + 5 = 18$
- $g(3, \{4\}) = c_{34} + g(4, \emptyset) = 12 + 8 = 20$
- $g(4, \{2\}) = c_{42} + g(2, \emptyset) = 8 + 5 = 13$
- $g(4, \{3\}) = c_{43} + g(3, \emptyset) = 9 + 6 = 15$

问题实例



	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

$|S|=2$

- $g(2, \{3, 4\}) = \min\{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\}$
 $= \min\{9 + 20, 10 + 15\} = 25$
- $g(3, \{2, 4\}) = \min\{c_{32} + g(2, \{4\}), c_{34} + g(4, \{2\})\}$
 $= \min\{13 + 18, 12 + 13\} = 25$
- $g(4, \{2, 3\}) = \min\{c_{42} + g(2, \{3\}), c_{43} + g(3, \{2\})\}$
 $= \min\{8 + 15, 9 + 18\} = 23$

$|S|=3$

- $g(1, \{2, 3, 4\}) = \min\{c_{12} + g(2, \{3, 4\}), c_{13} + g(3, \{2, 4\}), c_{14} + g(4, \{2, 3\})\}$
 $= \min\{10 + 25, 15 + 25, 20 + 23\} = 35$

可得这条最优周游路线是: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$

思考1: 从2出发的最优周游路线是那一条?

思考2: 和贪心法相比为什么能够保证获得最优解?

- $|S| = k$, $k = 0, 1, \dots, n-2$ 时, 对于 k 的每一种取值, $g(i, S)$ 中的节点 i 都有 $n-1$ 种选择, 即 $i=2, 3, \dots, n$, 对应每个 i , S 是 $V \setminus \{1, i\}$ 的子集, 个数为 C_{n-2}^k
- k 的每一种取值对应的函数求解次数为: $(n-1)C_{n-2}^k$
- k 的所有取值对应的函数求解次数为: $\sum_{k=0}^{n-2} (n-1)C_{n-2}^k = (n-1) \cdot 2^{n-2}$
- 求函数 $g(i, S)$ 的值要进行 $|S|-1$ 比较: $\sum_{k=0}^{n-2} (n-1)(k-1)C_{n-2}^k = \Theta(n^2 2^n)$

优于枚举法的结果 $O(n!)$,
但随着 n 的增大, 该方法亦不可取。

思考: 为何该问题的动态规划算法效率差?